
Rerank와 Modular RAG

3. Modular RAG

목차

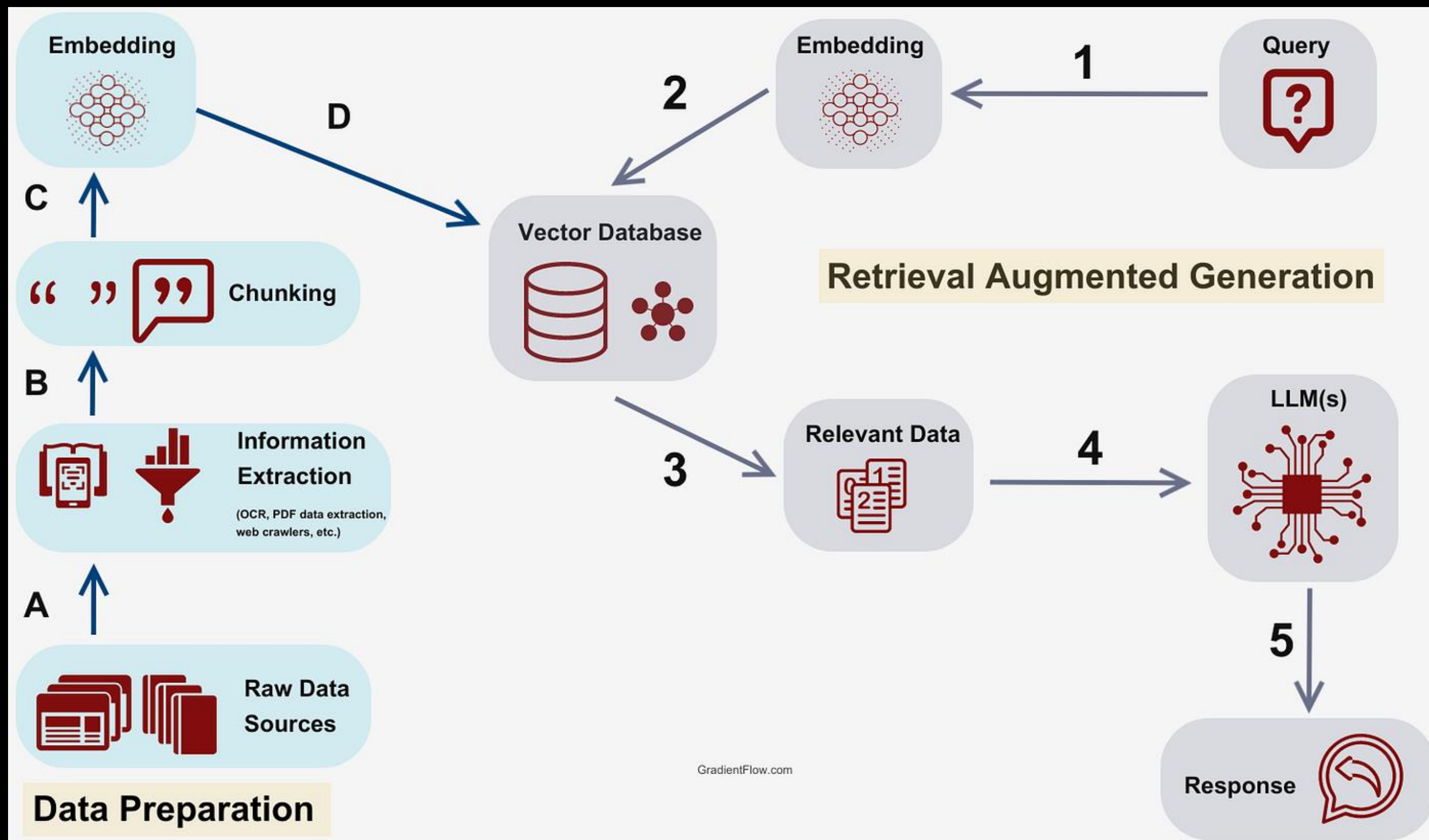
1. RAG Research-to-Production Gap & Advanced RAG
2. 3. Modular RAG

1. RAG Research-to-Production Gap & Advanced RAG



RAG 구조도

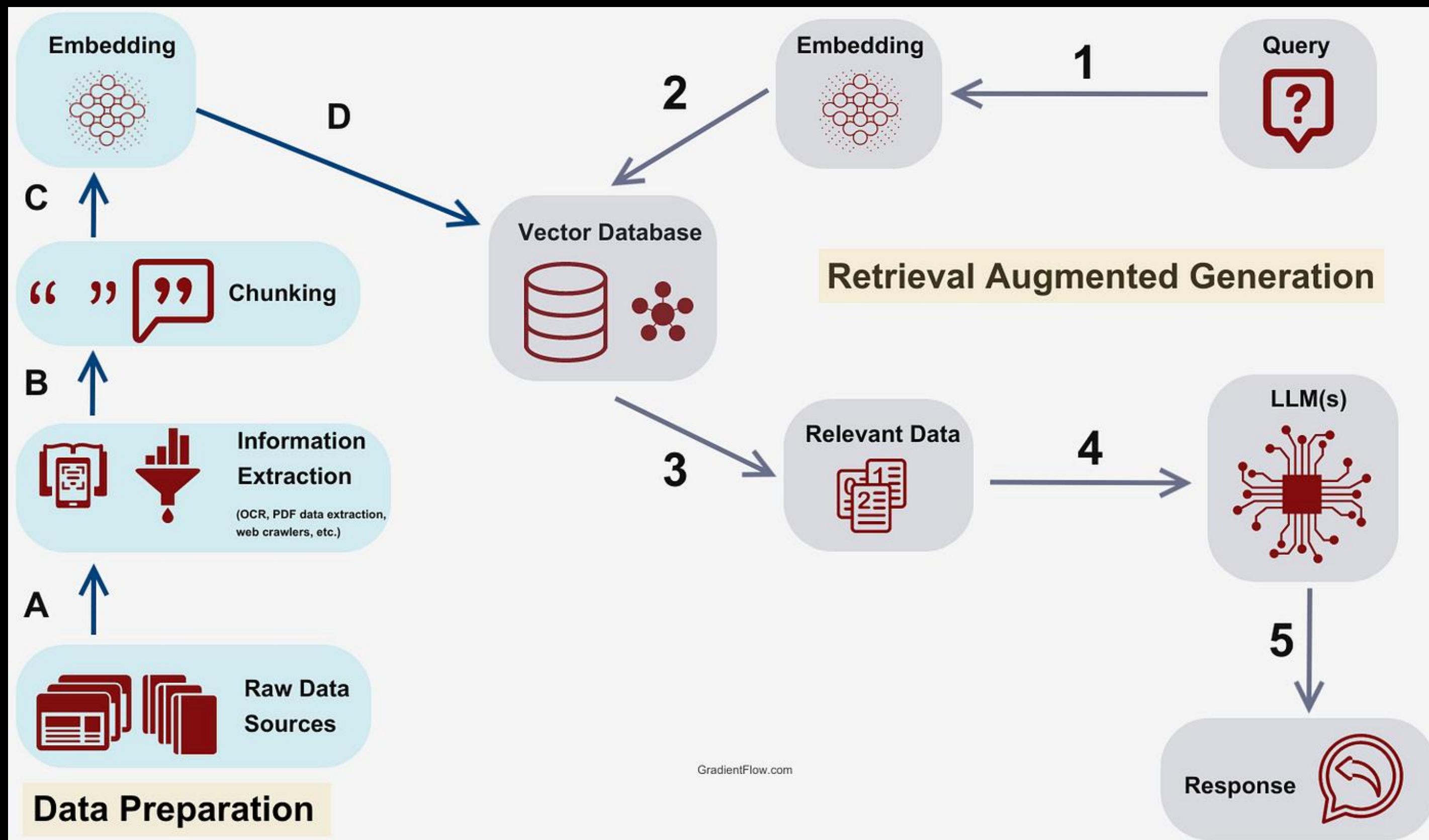
- Prep
 - Extraction
 - Chunking
 - Embedding
 - Vector DB loading





RAG 구조도

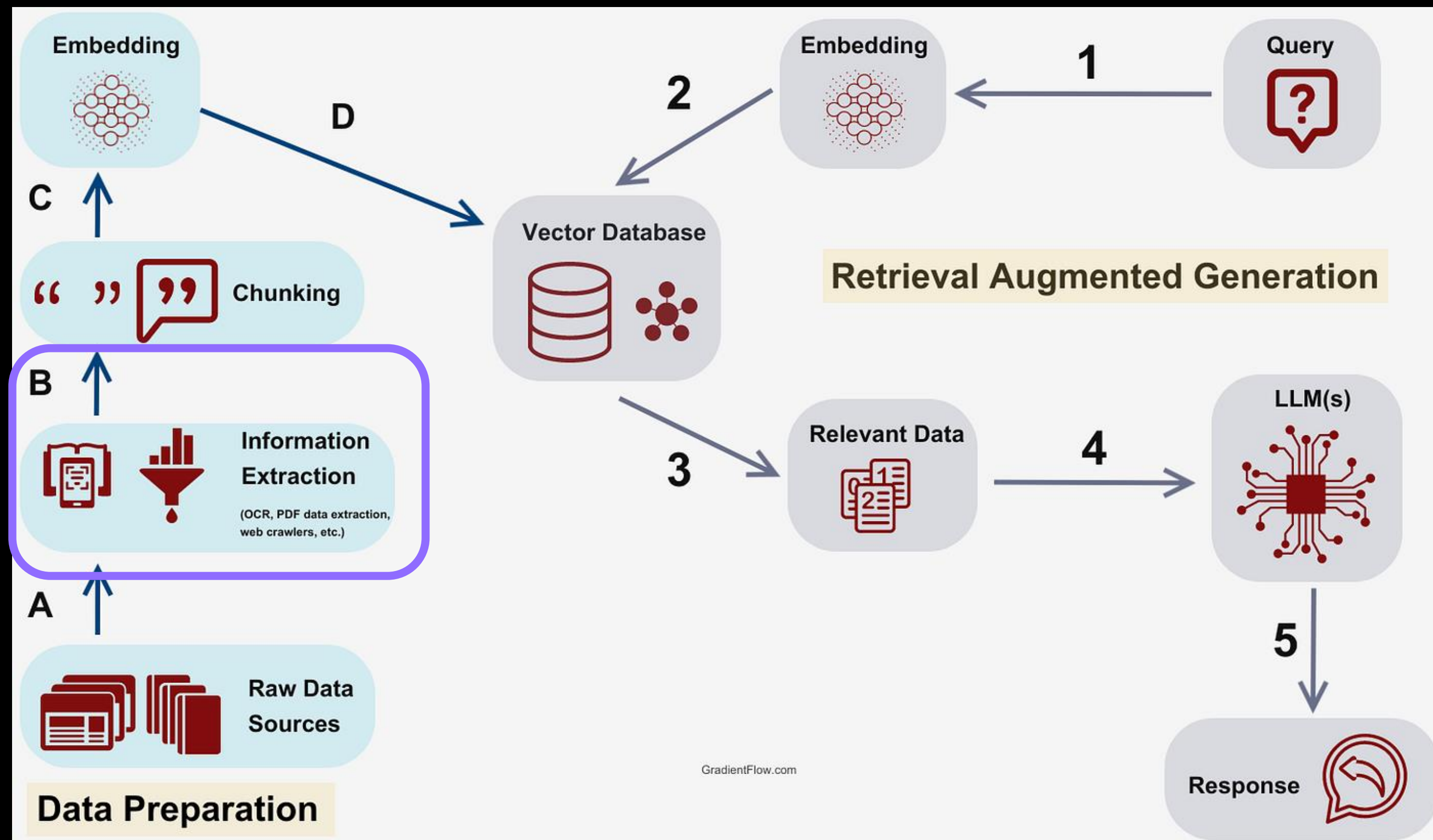
- RAG
 - Query embedding
 - Retrieval
 - Generation
 - Post-processing





Challenge – Extraction

- Prep
 - Extraction
 - Chunking
 - Embedding
 - Vector DB loading



- Document 내에는 다양한 모달리티가 포함

- 텍스트

● Ⅲ

- 이미지

- 그래프

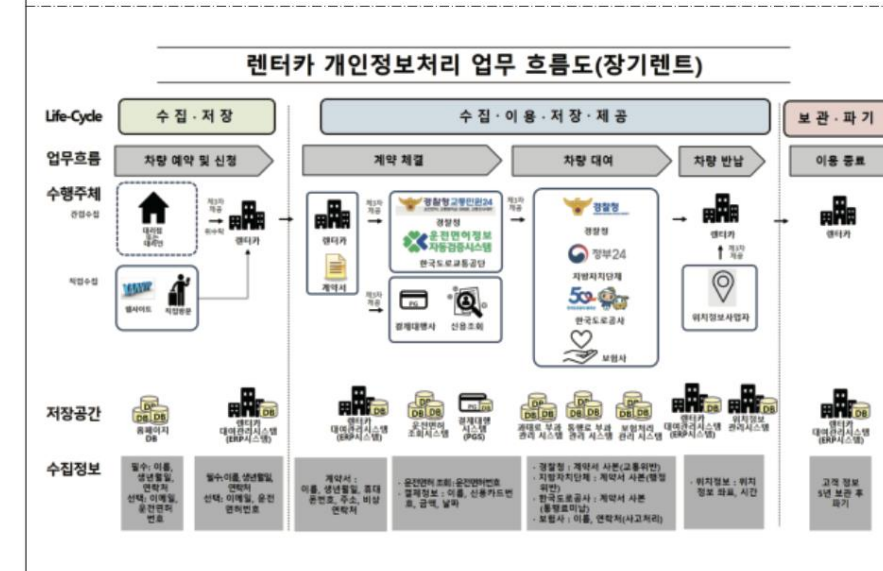
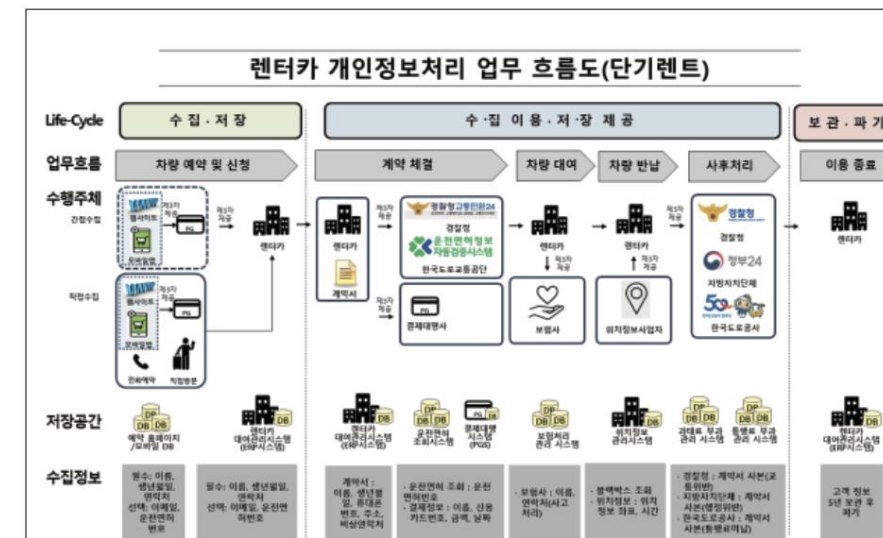
- 그러나 LLM은 텍스트만 이해 가능

- 이미지, 표, 그래프는 다른 방식 활용 필요

I. 렌터카 개인정보 처리

1. 렌터카 업무 개요

(1) 렌터카 업무 개인정보 흐름도



한국렌터카사업조합연합회 참여 회원사를 위한 렌터카 업무 개인정보 처리 가이드

- ▶ 온라인 사이트, 모바일 앱으로부터 제공받을 시 안전하게 제공 받도록 한다.
 - 시스템으로 전송할 경우, 암호화 전송하도록 함
 - 전자파일을 받을 경우는 파일을 암호화하여 받도록 함
 - ▶ 렌터카 회사는 제공 받은 고객 대상으로 아래의 3가지 사항을 알리고 난 후, 차량 예약 상담을 수행하여야 한다. 제3자로부터 개인정보를 제공받은 여행사가 5만 명 이상의 고유식별 정보를 처리하거나 100만 명 이상의 고객의 개인정보를 처리하는 렌터카 회사는 다음의 사항을 알리고 난 후, 차량 예약 상담을 하여야 한다.
- [정보주체 출처 고지 항목]

 - ① 개인정보의 수집 출처(예, OO쇼핑몰을 통해 수집됨)
 - ② 개인정보의 처리 목적(예, 여행 상담 서비스)
 - ③ 개인정보 처리의 정치를 요구할 권리가 있다는 사실
- ▶ 제공받은 정보는 예약 상담 목적 외로는 사용되지 않도록 하여야 한다.
 - ▶ 차량 예약 상담이 종료되면 해당 개인정보는 지체 없이 파기하도록 한다.

- 계약 체결이 되지 않는 경우 제공받은 개인정보의 파기

온라인 사이트, 모바일 앱으로부터 제공받은 고객정보를 바탕으로 차량 예약 상담 이후 예약 체결이 되지 않고 상담으로만 끝나는 경우라면 제공받은 고객 정보는 지체 없이(5일 이내) 파기하여야 한다.

(3) 당초 고객 개인정보 수집 목적과 합리적 목적 범위 내에서의 고객의 동의 없이 개인정보를 추가적으로 이용·제공할 수 있는 경우

렌터카 업체는 당초 수집 목적과 합리적으로 관련된 범위에서 정보주체에게 불이익이 발생하는지 여부, 암호화 등 안전성 확보에 필요한 조치를 하였는지 여부 등을 고려하여 정보주체의 동의 없이 개인정보를 이용·제공 가능하다.

이 때, 다음 사항을 고려하여야 한다.

- ① 개인정보를 추가적으로 이용·제공하려는 목적이 당초 수집 목적과 관련성이 있는지 여부
- ② 개인정보를 수집한 정황 또는 처리 관행에 비추어 볼 때 추가적인 이용·제공에 대한 예측 가능성이 있는지 여부
- ③ 개인정보의 추가적 이용·제공이 정보주체 이익을 부당하게 침해하는지 여부
- ④ 가명처리 또는 암호화 등 안전성 확보에 필요한 조치를 하였는지 여부



Challenge – Extraction

- PDF는 다양한 방식을 통해 생성
- 오류 또한 다양하게 발생
 - 생성된 애플리케이션에 의하여
 - 유니코드 에러
 - 폰트
 - 특수 기호
 - 문서의 디자인 등

[illegible]

대여요금을 각 적용합니다.

③고객은 제1항에 의한 대체 렌터카의 임차를 거절할 수 있으며, 이 경우 회사는 고객에게 예약금 전액을 반환합니다.

제6조(요금의 수령방법 등) ①고객은 원칙적으로 약정한 대여요금을 선납하여야 합니다. 다만, 회사와 고객은 대여요금을 분납하거나 사용 후에 지급하기로 별도의 약정을 할 수 있습니다.

②고객의 요구로 인하여 대여요금 외의 추가비용이 발생한 경우에는 고객은 그 추가비용을 부담하여야 합니다.

③고객은 임차기간을 초과하여 렌터카를 사용한 경우에는 렌터카 반환시추가로 대여요금을 지급하여야 합니다.

제7조(회사의 대여계약 해지) ①회사는 다음 각 호의 어느 하나에 해당하는 경우에는 계약을 해지할 수 있습니다.

1. 고객이 계약의 중요한 사항을 위반하여 계약을 유지하기 어려운 객관적인 사정이 존재할 때
2. 계약 당시 고객의 개인정보가 허위로 판명된 때
3. 대여요금을 분할납부하기로 한 경우로서 대여기간 중 차임연체액이 2기의 대여요금에 달한 때
4. 고객(고객 아닌 자가 임대차계약서상의 운전자인 경우에는 운전자를 말한다. 이하 이 항에서 같다)의 운전면허가 취소 또는 정지된 때
5. 고객이 교통사고를 야기한 때
6. 렌터카 대여 후 고객이 음주운전을 한 때
7. 렌터카 대여 후 고객이 마약, 각성제, 신나 등 약물에 취한 채 운전한 때
8. 임대차계약서상의 운전자 이외의 자가 운전을 한 때
9. 제15조의 금지행위를 한 때

②제1항에 따라 계약이 해지된 경우 회사는 잔여기간 대여요금의 10%를 공제한



Challenge – Extraction

- PDF는 다양한 방식을 통해 생성
- 오류 또한 다양하게 발생
 - 생성된 애플리케이션에 의하여
 - 유니코드 에러
 - 폰트
 - 특수 기호
 - 문서의 디자인 등

001 □ 70 문자

!"#\$%&'()*+,-.%/0\$!"#\$\$%&'()*+,-."%
1&'()*+*234%567%89%;;<='_`%>?

002 □ 168 문자

≡ # #
#ぞ1#'ゝ臣#♯于 ㉔㉕㉖㉗㉘㉙㉚㉛㉜㉝㉞㉟㊱㊲㊳㊴㊵㊶㊷㊸㊹㊺㊻㊼㊽㊾㊿
㏀㏁㏂㏃㏄㏅㏆㏇㏈㏉㏐㏑㏒㏓㏔㏕㏖㏗㏘㏙㏚㏛㏜㏝㏞㏟㏠㏡㏢㏣㏤㏥㏦㏧㏨㏩㏪㏫㏬㏭㏮㏯㏰㏱㏲㏳㏴㏵㏶㏷㏸㏹㏺㏻㏼㏽㏾㏿
㐀㐁㐂㐃㐄㐅㐆㐇㐈㐉㐊㐋㐌㐍㐎㐏㐐㐑㐒㐓㐔㐕㐖㐗㐘㐙㐚㐛㐜㐝㐞㐟㐠㐡㐢㐣㐤㐥㐦㐧㐨㐩㐪㐫㐬㐭㐮㐯㐰㐱㐲㐳㐴㐵㐶㐷㐸㐹㐺㐻㐼㐽㐾㐿
㑀㑁㑂㑃㑄㑅㑆㑇㑈㑉㑊㑋㑌㑍㑎㑏㑐㑑㑒㑓㑔㑕㑖㑗㑘㑙㑚㑛㑜㑝㑞㑟㑠㑡㑢㑣㑤㑥㑦㑧㑨㑩㑪㑫㑬㑭㑮㑯㑰㑱㑲㑳㑴㑵㑶㑷㑸㑹㑺㑻㑼㑽㑾㑿
㒀㒁㒂㒃㒄㒅㒆㒇㒈㒉㒊㒋㒌㒍㒎㒏㒐㒑㒒㒓㒔㒕㒖㒗㒘㒙㒚㒛㒜㒝㒞㒟㒠㒡㒢㒣㒤㒥㒦㒧㒨㒩㒪㒫㒬㒭㒮㒯㒰㒱㒲㒳㒴㒵㒶㒷㒸㒹㒺㒻㒼㒽㒾㒿
㓀㓁㓂㓃㓄㓅㓆㓇㓈㓉㓊㓋㓌㓍㓎㓏㓐㓑㓒㓓㓔㓕㓖㓗㓘㓙㓚㓛㓜㓝㓞㓟㓠㓡㓢㓣㓤㓥㓦㓧㓨㓩㓪㓫㓬㓭㓮㓯㓰㓱㓲㓳㓴㓵㓶㓷㓸㓹㓺㓻㓼㓽㓾㓿
㔀㔁㔂㔃㔄㔅㔆㔇㔈㔉㔊㔋㔌㔍㔎㔏㔐㔑㔒㔓㔔㔕㔖㔗㔘㔙㔚㔛㔜㔝㔞㔟㔠㔡㔢㔣㔤㔥㔦㔧㔨㔩㔪㔫㔬㔭㔮㔯㔰㔱㔲㔳㔴㔵㔶㔷㔸㔹㔺㔻㔼㔽㔾㔿
㕀㕁㕂㕃㕄㕅㕆㕇㕈㕉㕊㕋㕌㕍㕎㕏㕐㕑㕒㕓㕔㕕㕖㕗㕘㕙㕚㕛㕜㕝㕞㕟㕠㕡㕢㕣㕤㕥㕦㕧㕨㕩㕪㕫㕬㕭㕮㕯㕰㕱㕲㕳㕴㕵㕶㕷㕸㕹㕺㕻㕼㕽㕾㕿
㖀㖁㖂㖃㖄㖅㖆㖇㖈㖉㖊㖋㖌㖍㖎㖏㖐㖑㖒㖓㖔㖕㖖㖗㖘㖙㖚㖛㖜㖝㖞㖟㖠㖡㖢㖣㖤㖥㖦㖧㖨㖩㖪㖫㖬㖭㖮㖯㖰㖱㖲㖳㖴㖵㖶㖷㖸㖹㖺㖻㖼㖽㖾㖿
㗀㗁㗂㗃㗄㗅㗆㗇㗈㗉㗊㗋㗌㗍㗎㗏㗐㗑㗒㗓㗔㗕㗖㗗㗘㗙㗚㗛㗜㗝㗞㗟㗠㗡㗢㗣㗤㗥㗦㗧㗨㗩㗪㗫㗬㗭㗮㗯㗰㗱㗲㗳㗴㗵㗶㗷㗸㗹㗺㗻㗼㗽㗾㗿
㘀㘁㘂㘃㘄㘅㘆㘇㘈㘉㘊㘋㘌㘍㘎㘏㘐㘑㘒㘓㘔㘕㘖㘗㘘㘙㘚㘛㘜㘝㘞㘟㘠㘡㘢㘣㘤㘥㘦㘧㘨㘩㘪㘫㘬㘭㘮㘯㘰㘱㘲㘳㘴㘵㘶㘷㘸㘹㘺㘻㘼㘽㘾㘿
㙀㙁㙂㙃㙄㙅㙆㙇㙈㙉㙊㙋㙌㙍㙎㙏㙐㙑㙒㙓㙔㙕㙖㙗㙘㙙㙚㙛㙜㙝㙞㙟㙠㙡㙢㙣㙤㙥㙦㙧㙨㙩㙪㙫㙬㙭㙮㙯㙰㙱㙲㙳㙴㙵㙶㙷㙸㙹㙺㙻㙼㙽㙾㙿
㚀㚁㚂㚃㚄㚅㚆㚇㚈㚉㚊㚋㚌㚍㚎㚏㚐㚑㚒㚓㚔㚕㚖㚗㚘㚙㚚㚛㚜㚝㚞㚟㚠㚡㚢㚣㚤㚥㚦㚧㚨㚩㚪㚫㚬㚭㚮㚯㚰㚱㚲㚳㚴㚵㚶㚷㚸㚹㚺㚻㚼㚽㚾㚿
㛀㛁㛂㛃㛄㛅㛆㛇㛈㛉㛊㛋㛌㛍㛎㛏㛐㛑㛒㛓㛔㛕㛖㛗㛘㛙㛚㛛㛜㛝㛞㛟㛠㛡㛢㛣㛤㛥㛦㛧㛨㛩㛪㛫㛬㛭㛮㛯㛰㛱㛲㛳㛴㛵㛶㛷㛸㛹㛺㛻㛼㛽㛾㛿
㜀㜁㜂㜃㜄㜅㜆㜇㜈㜉㜊㜋㜌㜍㜎㜏㜐㜑㜒㜓㜔㜕㜖㜗㜘㜙㜚㜛㜜㜝㜞㜟㜠㜡㜢㜣㜤㜥㜦㜧㜨㜩㜪㜫㜬㜭㜮㜯㜰㜱㜲㜳㜴㜵㜶㜷㜸㜹㜺㜻㜼㜽㜾㜿
㝀㝁㝂㝃㝄㝅㝆㝇㝈㝉㝊㝋㝌㝍㝎㝏㝐㝑㝒㝓㝔㝕㝖㝗㝘㝙㝚㝛㝜㝝㝞㝟㝠㝡㝢㝣㝤㝥㝦㝧㝨㝩㝪㝫㝬㝭㝮㝯㝰㝱㝲㝳㝴㝵㝶㝷㝸㝹㝺㝻㝼㝽㝾㝿
㞀㞁㞂㞃㞄㞅㞆㞇㞈㞉㞊㞋㞌㞍㞎㞏㞐㞑㞒㞓㞔㞕㞖㞗㞘㞙㞚㞛㞜㞝㞞㞟㞠㞡㞢㞣㞤㞥㞦㞧㞨㞩㞪㞫㞬㞭㞮㞯㞰㞱㞲㞳㞴㞵㞶㞷㞸㞹㞺㞻㞼㞽㞾㞿
㟀㟁㟂㟃㟄㟅㟆㟇㟈㟉㟊㟋㟌㟍㟎㟏㟐㟑㟒㟓㟔㟕㟖㟗㟘㟙㟚㟛㟜㟝㟞㟟㟠㟡㟢㟣㟤㟥㟦㟧㟨㟩㟪㟫㟬㟭㟮㟯㟰㟱㟲㟳㟴㟵㟶㟷㟸㟹㟺㟻㟼㟽㟾㟿
㠀㠁㠂㠃㠄㠅㠆㠇㠈㠉㠊㠋㠌㠍㠎㠏㠐㠑㠒㠓㠔㠕㠖㠗㠘㠙㠚㠛㠜㠝㠞㠟㠠㠡㠢㠣㠤㠥㠦㠧㠨㠩㠪㠫㠬㠭㠮㠯㠰㠱㠲㠳㠴㠵㠶㠷㠸㠹㠺㠻㠼㠽㠾㠿
㡀㡁㡂㡃㡄㡅㡆㡇㡈㡉㡊㡋㡌㡍㡎㡏㡐㡑㡒㡓㡔㡕㡖㡗㡘㡙㡚㡛㡜㡝㡞㡟㡠㡡㡢㡣㡤㡥㡦㡧㡨㡩㡪㡫㡬㡭㡮㡯㡰㡱㡲㡳㡴㡵㡶㡷㡸㡹㡺㡻㡼㡽㡾㡿
㢀㢁㢂㢃㢄㢅㢆㢇㢈㢉㢊㢋㢌㢍㢎㢏㢐㢑㢒㢓㢔㢕㢖㢗㢘㢙㢚㢛㢜㢝㢞㢟㢠㢡㢢㢣㢤㢥㢦㢧㢨㢩㢪㢫㢬㢭㢮㢯㢰㢱㢲㢳㢴㢵㢶㢷㢸㢹㢺㢻㢼㢽㢾㢿
㣀㣁㣂㣃

대여요금을 각 적용합니다.

③고객은 제1항에 의한 대체 렌터카의 임차를 거절할 수 있으며, 이 경우 회사는 고객에게 예약금 전액을 반환합니다.

제6조(요금의 수령방법 등) ①고객은 원칙적으로 약정한 대여요금을 선납하여야 합니다. 다만, 회사와 고객은 대여요금을 분납하거나 사용 후에 지급하기로 별도의 약정을 할 수 있습니다.

②고객의 요구로 인하여 대여요금 외의 추가비용이 발생한 경우에는 고객은 그 추가비용을 부담하여야 합니다.

③고객은 임차기간을 초과하여 렌터카를 사용한 경우에는 렌터카 반환시추가로 대여요금을 지급하여야 합니다.

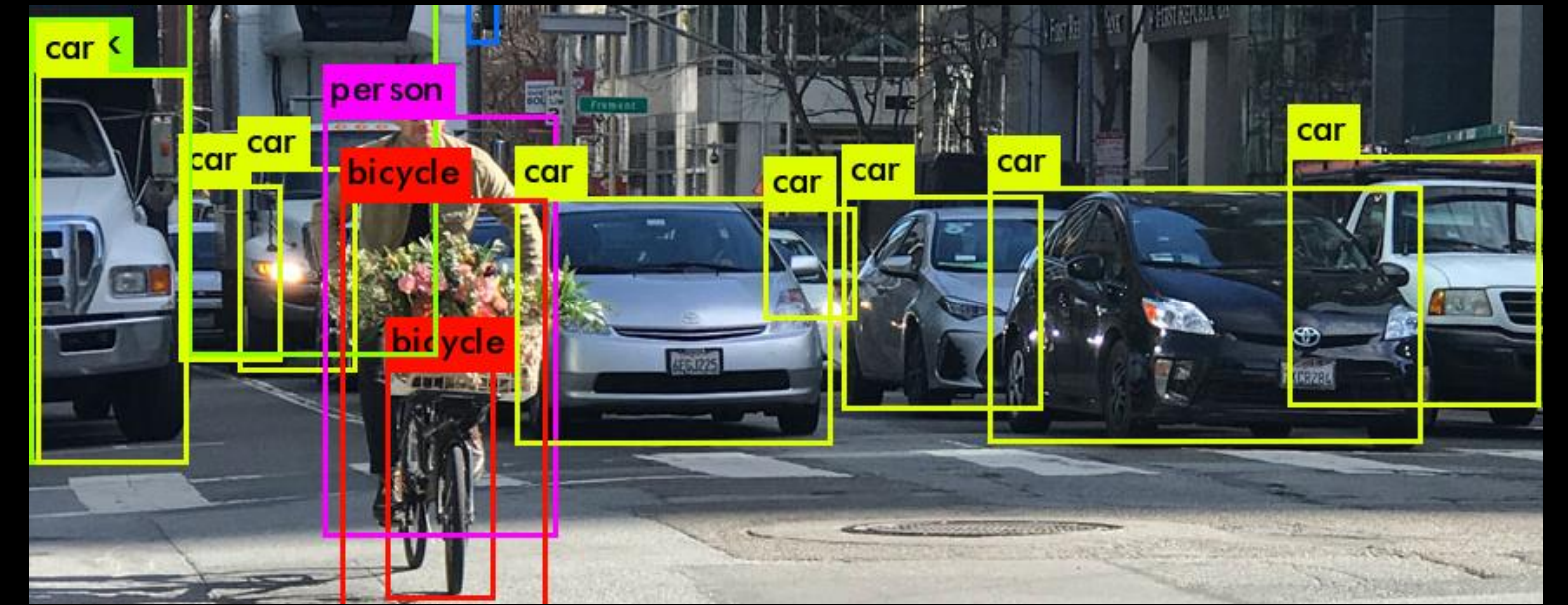
제7조(회사의 대여계약 해지) ①회사는 다음 각 호의 어느 하나에 해당하는 경우에는 계약을 해지할 수 있습니다.

1. 고객이 계약의 중요한 사항을 위반하여 계약을 유지하기 어려운 객관적인 사정이 존재할 때
2. 계약 당시 고객의 개인정보가 허위로 판명된 때
3. 대여요금을 분할납부하기로 한 경우로서 대여기간 중 차임연체액이 2기의 대여요금에 달한 때
4. 고객(고객 아닌 자가 임대차계약서상의 운전자인 경우에는 운전자를 말한다. 이하 이 항에서 같다)의 운전면허가 취소 또는 정지된 때
5. 고객이 교통사고를 야기한 때
6. 렌터카 대여 후 고객이 음주운전을 한 때
7. 렌터카 대여 후 고객이 마약, 각성제, 신나 등 약물에 취한 채 운전한 때
8. 임대차계약서상의 운전자 이외의 자가 운전을 한 때
9. 제15조의 금지행위를 한 때

②제1항에 따라 계약이 해지된 경우 회사는 잔여기간 대여요금의 10%를 공제한

Advanced RAG – Extraction

- OCR(Optical Character Recognition)
 - 광학 문자 인식 기술
 - 인쇄된 문서를 디지털 이미지 파일로 변환하는 기술
 - 스마트폰 카메라, 카카오톡, 구글 번역기, 대부분의 PDF 편집기에 내재
- 다만 텍스트를 인식하기 위해선, 우선 텍스트가 존재하는 영역을 인식해야 함
 - Document detection 기술 활용
 - Object detection 기술을 문서 인식에 활용



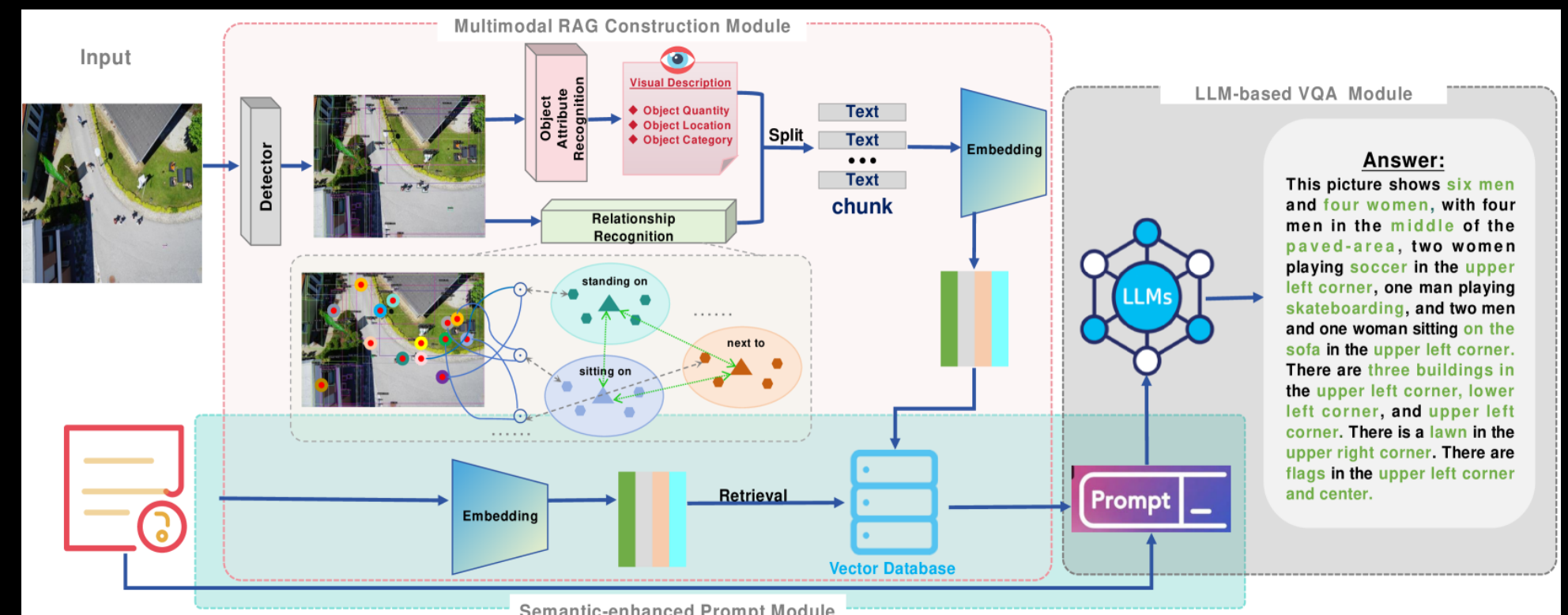
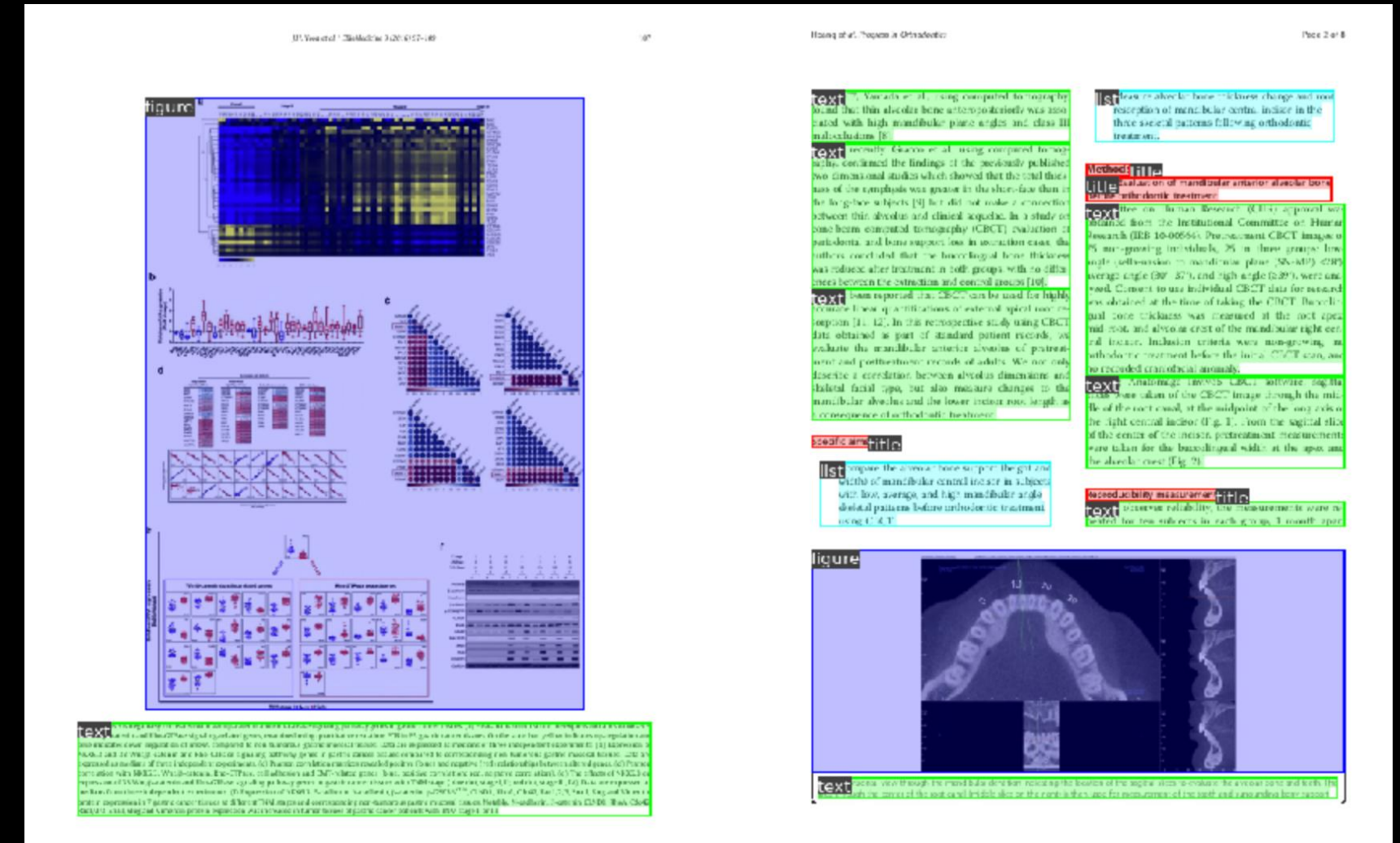
Original Document	Apply Object Detection
12 가계신용 레버리지는 축소된 반면 기업신용 레버리지는 확대 지속 부문별로 보면, 가계신용 레버리지는 축소된 반면 기업신용 레버리지는 계속 확대되는 모습이다. 2023년 1/4분기말 가계신용/명목GDP 비율은 103.4%로 2022년 3/4분기말 104.8%에서 1.4%포인트 하락하였으나, 기업신용/명목GDP 비율은 같은 기간중 118.7%에서 119.7%로 1.0%포인트 높아졌다. 가계신용은 부동산경기 둔화, 대출금리 상승 등의 영향으로 증가세가 크게 둔화되었으나, 기업신용은 은행들의 대출 확대 노력 및 회사채 순발행 지속 등으로 꾸준한 증가세를 이어갔다(그림 1-3). 그림 1-3. 부문별 신용 레버리지 및 신용 증가율[※] 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 주: 1) 지급은행별 기준(2022년 14분기)은 추정치 2) 전산분기자료 자료: 한국은행 가계신용 레버리지는 축소된 반면 기업신용 레버리지는 확대 지속 가계신용 레버리지가 장기추세를 회피하는 폭이 확대되었다. 가계신용/명목GDP 증은 2022년 3/4분기 마이너스로 전환(0.3%포인트)된 이후 2023년 1/4분기에 -2.9%포인트로 확대되었다. 한편 기업신용 레버리지는 장기추세를 웃도는 상황이 지속되고 있으나, 상회하는 정도는 축소되었다. 기업신용/명목GDP 증은 2023년 1/4분기 +6.1%포인트로 2022년 3/4분기(+7.9%포인트)에 비해서 줄어들었다(그림 1-4). 가계신용은 부동산경기 둔화, 대출금리 상승 등의 영향으로 증가세가 크게 둔화되었으나, 기업신용은 은행들의 대출 확대 노력 및 회사채 순발행 지속 등으로 꾸준한 증가세를 이어갔다(그림 1-3). 그림 1-4. 부문별 민간신용/명목GDP 비율 및 점[※] 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 주: 1) 지급은행별 기준(2022년 14분기)은 추정치 2) 전산분기자료 자료: 한국은행 출처 : 금융안정보고서	12 가계신용 레버리지는 축소된 반면 기업신용 레버리지는 확대 지속 부문별로 보면, 가계신용 레버리지는 축소된 반면 기업신용 레버리지는 계속 확대되는 모습이다. 2023년 1/4분기말 가계신용/명목GDP 비율은 103.4%로 2022년 3/4분기말 104.8%에서 1.4%포인트 하락하였으나, 기업신용/명목GDP 비율은 같은 기간중 118.7%에서 119.7%로 1.0%포인트 높아졌다. 가계신용은 부동산경기 둔화, 대출금리 상승 등의 영향으로 증가세가 크게 둔화되었으나, 기업신용은 은행들의 대출 확대 노력 및 회사채 순발행 지속 등으로 꾸준한 증가세를 이어갔다(그림 1-3). 그림 1-3. 부문별 신용 레버리지 및 신용 증가율[※] 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 주: 1) 지급은행별 기준(2022년 14분기)은 추정치 2) 전산분기자료 자료: 한국은행 가계신용 레버리지는 축소된 반면 기업신용 레버리지는 확대 지속 가계신용 레버리지가 장기추세를 회피하는 폭이 확대되었다. 가계신용/명목GDP 증은 2022년 3/4분기 마이너스로 전환(0.3%포인트)된 이후 2023년 1/4분기에 -2.9%포인트로 확대되었다. 한편 기업신용 레버리지는 장기추세를 웃도는 상황이 지속되고 있으나, 상회하는 정도는 축소되었다. 기업신용/명목GDP 증은 2023년 1/4분기 +6.1%포인트로 2022년 3/4분기(+7.9%포인트)에 비해서 줄어들었다(그림 1-4). 가계신용은 부동산경기 둔화, 대출금리 상승 등의 영향으로 증가세가 크게 둔화되었으나, 기업신용은 은행들의 대출 확대 노력 및 회사채 순발행 지속 등으로 꾸준한 증가세를 이어갔다(그림 1-3). 그림 1-4. 부문별 민간신용/명목GDP 비율 및 점[※] 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 가계신용(명목GDP(좌축)) 기업신용(명목GDP(좌축)) 가계신용 증가율(우축) 기업신용 증가율(우축) 주: 1) 지급은행별 기준(2022년 14분기)은 추정치 2) 전산분기자료 자료: 한국은행 출처 : 금융안정보고서

<https://medium.com/@pedroazevedo6/object-detection-state-of-the-art-2022-ad750e0f6003>

<https://www.allganize.ai/ko/blog-posts-ko/https-blog-ko-allganize-ai-retrieval-augmented-generation-rag-reduce-hallucinations-enterprise-ai-2>

Advanced RAG – Extraction

- 이미지, 그래프의 경우 별도의 Logic 필요
- 해당 영역 또한 Document detection으로 영역 분리
- Bounding box를 처리할 수 있는 모델 필요
 - 이미지 인식 모델을 통해 이해 및 LLM과 융합
- Multimodal LLM을 활용하거나
- RAG에 LLM과 Vision model을 합치는 과정 필요



<https://www.allganize.ai/ko/blog-posts-ko/https-blog-ko-allganize-ai-retrieval-augmented-generation-rag-reduce-hallucinations-enterprise-ai-2>

<https://arxiv.org/html/2412.20927v1>

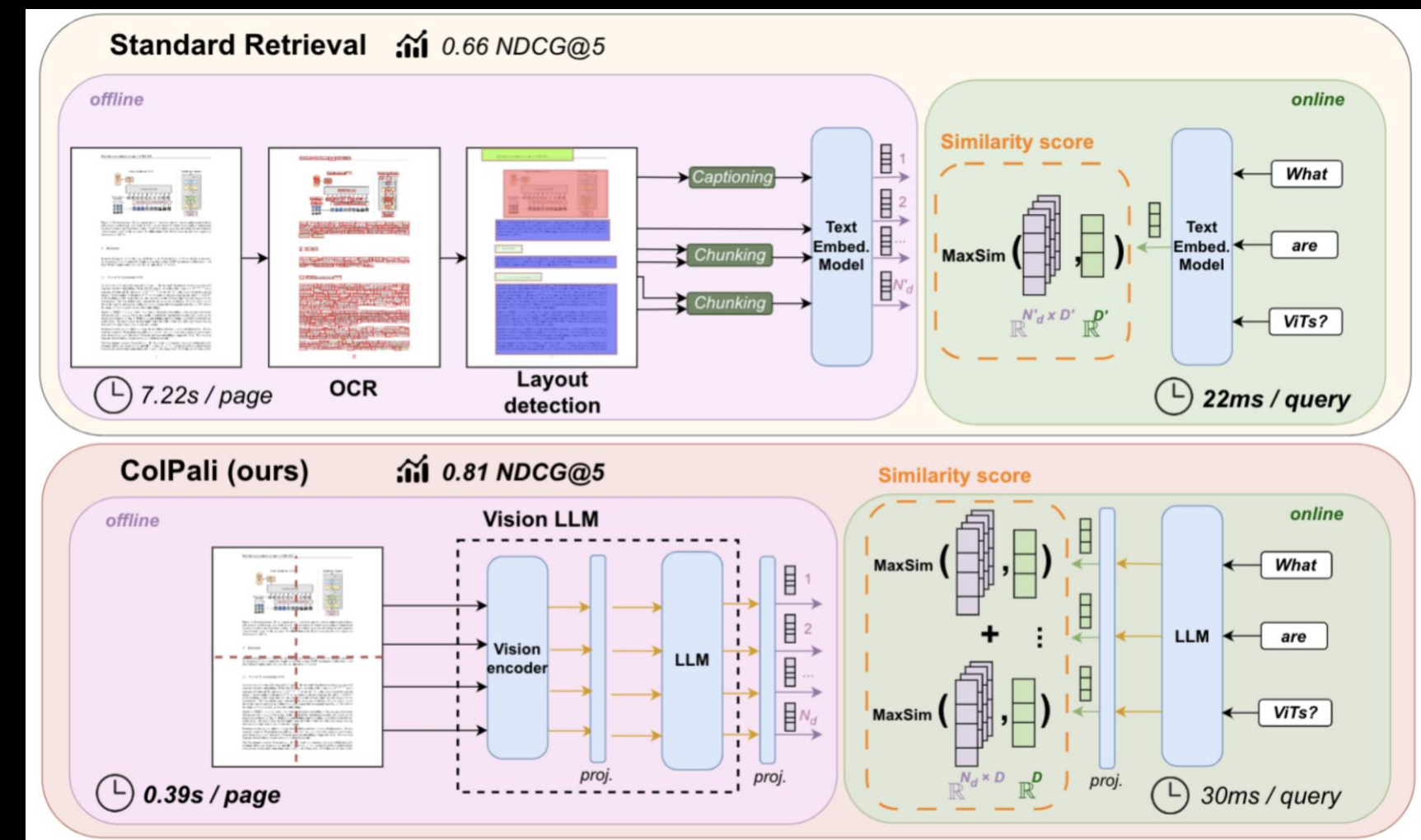


Advanced RAG – Extraction

- Document page embedding with VLM(Vision-Language Model)
 - Document detection을 사용할 경우
 - 제대로 텍스트가 OCR로 인식되지 않거나(4 ↔ 나)
 - 이미지, 그래프 등을 문서의 내용과 연관시키지 못할 확률이 존재
 - 그러나 Multimodal LLM 성능이 매우 향상
 - 특히 LLM의 Reasoning 역량이 매우 향상되었으며
 - 텍스트 뿐만 아니라 이미지, 그래프 등 다양한 데이터를 학습하였기에 이해할 수 있음
 - 또한 이미지 데이터를 바탕으로 답변할 수 있는 Trans-modality 성능 또한 우수

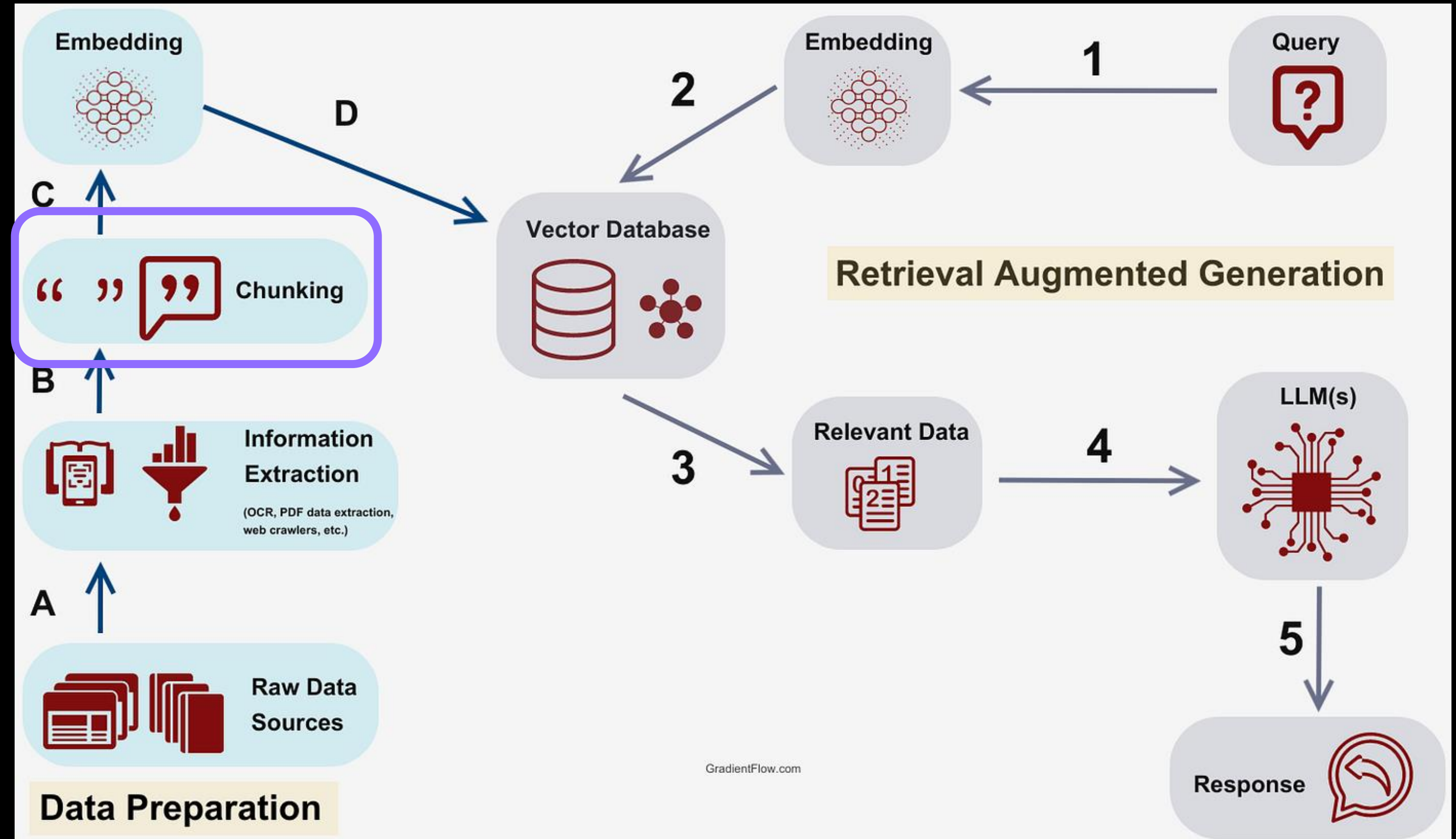
Advanced RAG – Extraction

- Document page embedding with VLM(Vision-Language Model)
- VLM을 활용하여
 - 문서를 이미지로 인식하고, 이를 페이지 단위로 Embedding 처리
 - Embedding 된 문서 내용을 기반으로 바로 답변 생성
 - 즉 LLM과 Embedding model이 하나로 결합되어 활용 가능
- ColPali(ICLR 2025) 등의 모델



Challenge – Chunking

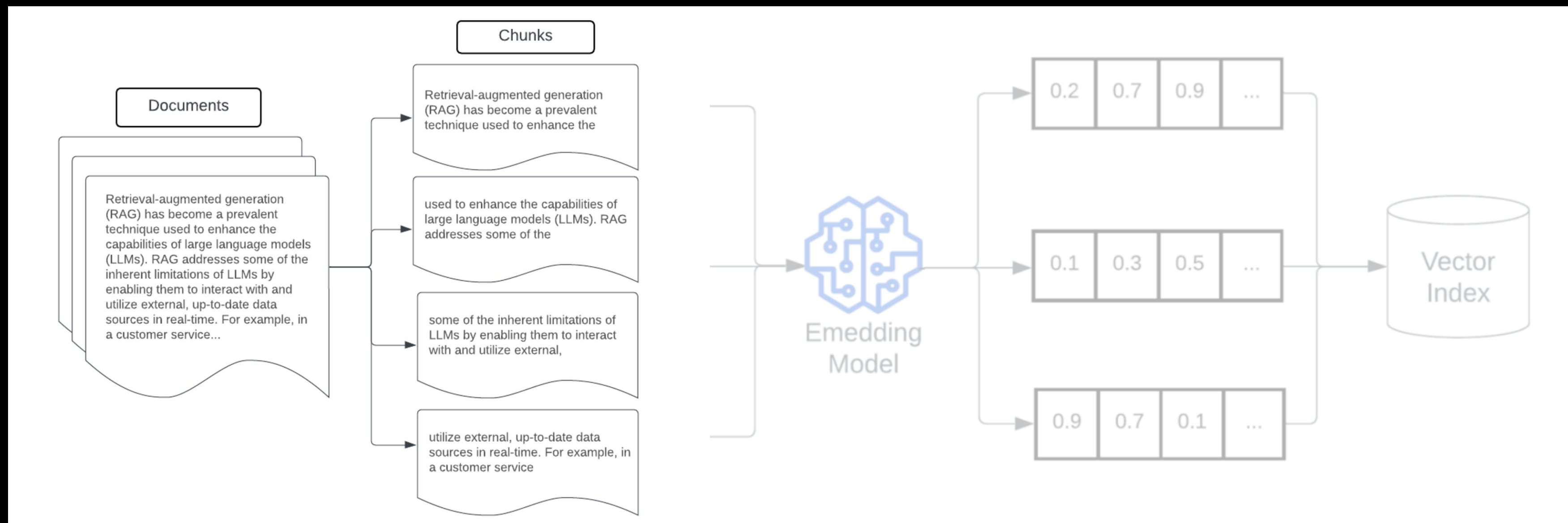
- Prep
 - Extraction
 - **Chunking**
 - Embedding
 - Vector DB loading





Challenge – Chunking

- 대규모 데이터 파일을 더 작은 단위로 나누는 과정
 - LLM이 필요한 정보만 정확하게 처리할 수 있도록
 - 특정 작업에 필요한 정보만 모델에 제공
 - 그러나 모든 문서에 동일한 기준으로 Chunking 진행 시 Semantic loss 발생(특수문자, 표, 그림 등에 의하여)



Tips – Chunking

- 다양한 Chunking strategy
 - 고정 크기 분할(Fixed-size): 지정된 문자 수만큼 분할하므로 단순하지만 문단이 훼손될 수 있음
 - 재귀적 분할(Recursive): 텍스트의 구조를 유지하는 선 안에서, 마침표 뿐만 아니라 여러 분할자("\n", ",", " " 등)를 사용하여 텍스트를 나눔
 - 문서 기반 분할(Document based): 문서에 마크다운, 코드, 테이블 등이 포함된 경우 구조에 맞게 텍스트를 분할하는 방법
 - 의미 기반 분할(Semantic): 문장 사이의 Embedding 유사도를 측정하여 분할 경계를 설정하는 방법

Four score and seven years ago our fathers brought forth on this continent, a new nation, conceived in Liberty, and dedicated to the proposition that all men are created equal. 1 2

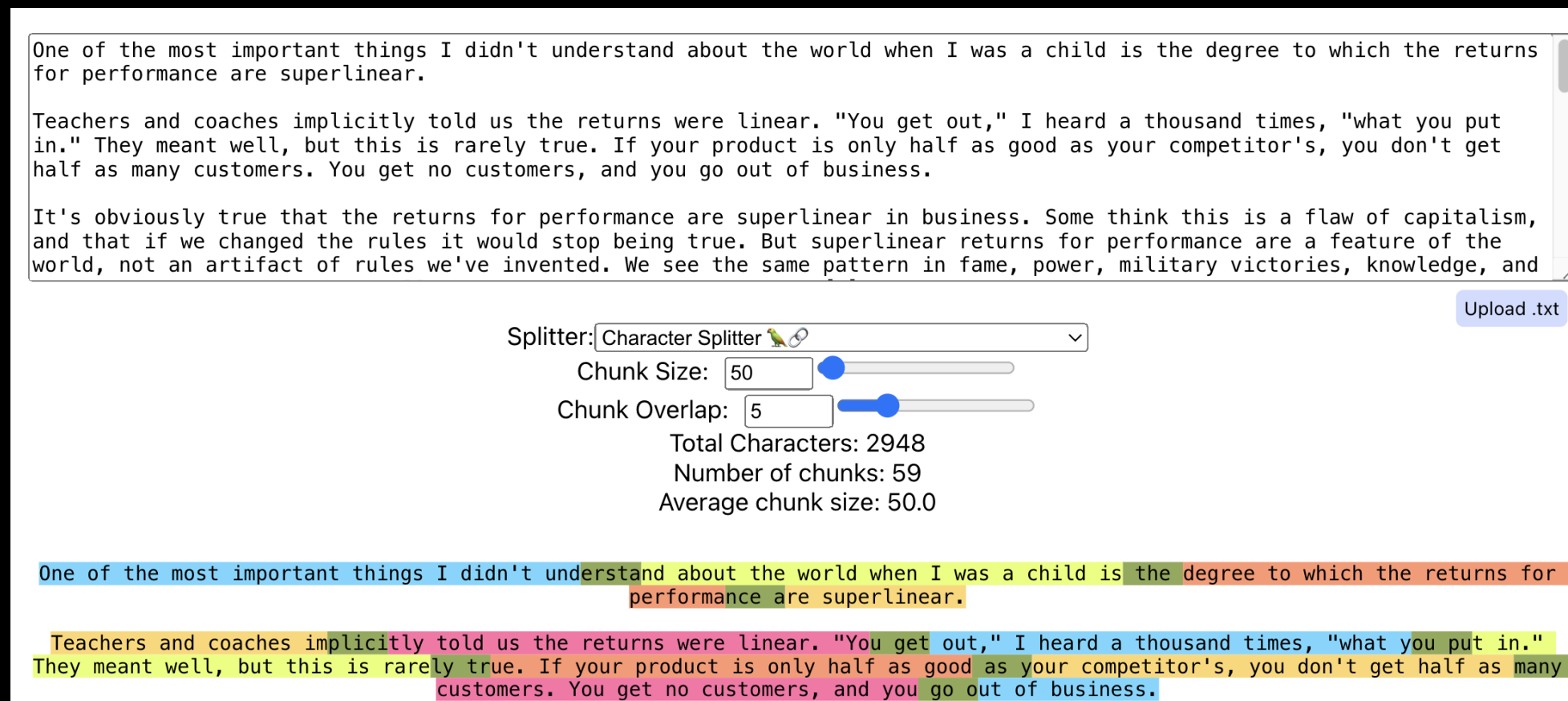
Now we are engaged in a great civil war, testing whether that nation, or any nation so conceived and so dedicated, can long endure. We are met on a great battle-field of that war. We have come to dedicate a portion of that field, as a final resting place for those who here gave their lives that that nation might live. It is altogether fitting and proper that we should do this. 3

Four score and seven years ago our fathers brought forth on this continent, a new nation, conceived in Liberty, and dedicated to the proposition that all men are created equal. 1 2

Now we are engaged in a great civil war, testing whether that nation, or any nation so conceived and so dedicated, can long endure. We are met on a great battle-field of that war. We have come to dedicate a portion of that field, as a final resting place for those who here gave their lives that that nation might live. It is altogether fitting and proper that we should do this. 3

Tips – Chunking

- Chunk overlapping
 - 문서를 분할할 때 문장, 문단, 혹은 챕터 안에서 문서가 분할될 경우 맥락이 단절될 수 있음
 - 이 경우 찾고자 하는 내용이 여러 Chunk에 나뉘거나, 다른 맥락의 텍스트와 묶이게 됨
 - 문서 분할 시 **중복을 허용**한다면 글의 흐름이 끊기거나, 검색결과에서 원하지 않는 내용이 포함될 가능성 감소
 - 다만 지나치게 많이 중복을 허용할 경우
 - 벡터DB의 크기가 커지고, 쿼리에 대하여 유사도가 높은 문장이 지나치게 많이 검색됨



The screenshot displays the chunkviz.up.railway.app interface. At the top, a text area contains three paragraphs of text. Below the text area, there are controls for splitting the text: a dropdown menu set to 'Character Splitter', a 'Chunk Size' slider set to 50, and a 'Chunk Overlap' slider set to 5. To the right of these controls is an 'Upload .txt' button. Below the sliders, the following statistics are shown: 'Total Characters: 2948', 'Number of chunks: 59', and 'Average chunk size: 50.0'. At the bottom, the text is visualized as a series of colored blocks representing the chunks, with some blocks overlapping to illustrate the 'Chunk Overlap' setting.

One of the most important things I didn't understand about the world when I was a child is the degree to which the returns for performance are superlinear.

Teachers and coaches implicitly told us the returns were linear. "You get out," I heard a thousand times, "what you put in." They meant well, but this is rarely true. If your product is only half as good as your competitor's, you don't get half as many customers. You get no customers, and you go out of business.

It's obviously true that the returns for performance are superlinear in business. Some think this is a flaw of capitalism, and that if we changed the rules it would stop being true. But superlinear returns for performance are a feature of the world, not an artifact of rules we've invented. We see the same pattern in fame, power, military victories, knowledge, and

Splitter: Character Splitter

Chunk Size: 50

Chunk Overlap: 5

Total Characters: 2948
Number of chunks: 59
Average chunk size: 50.0

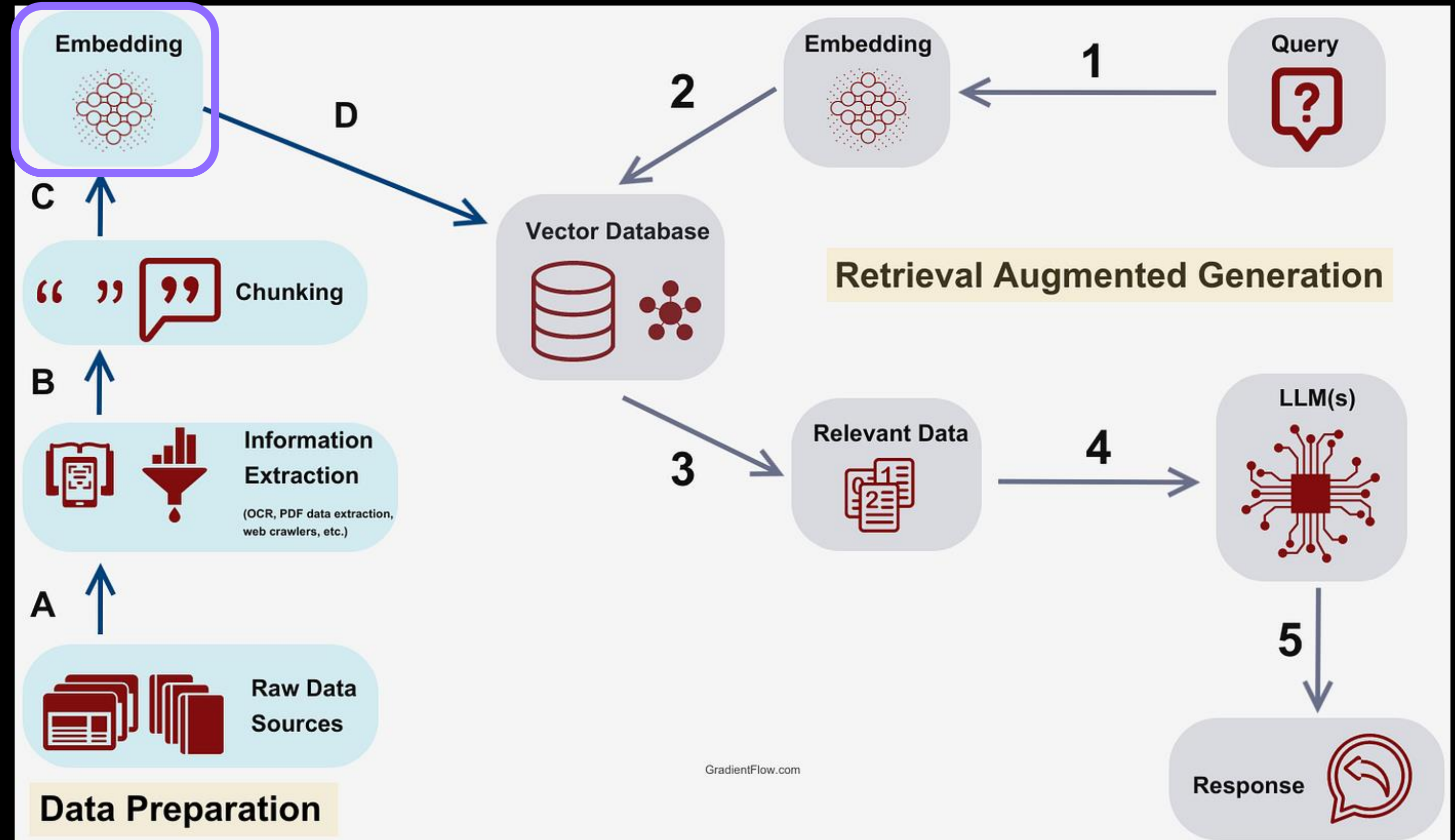
One of the most important things I didn't understand about the world when I was a child is the degree to which the returns for performance are superlinear.

Teachers and coaches implicitly told us the returns were linear. "You get out," I heard a thousand times, "what you put in." They meant well, but this is rarely true. If your product is only half as good as your competitor's, you don't get half as many customers. You get no customers, and you go out of business.



Challenge – Embedding

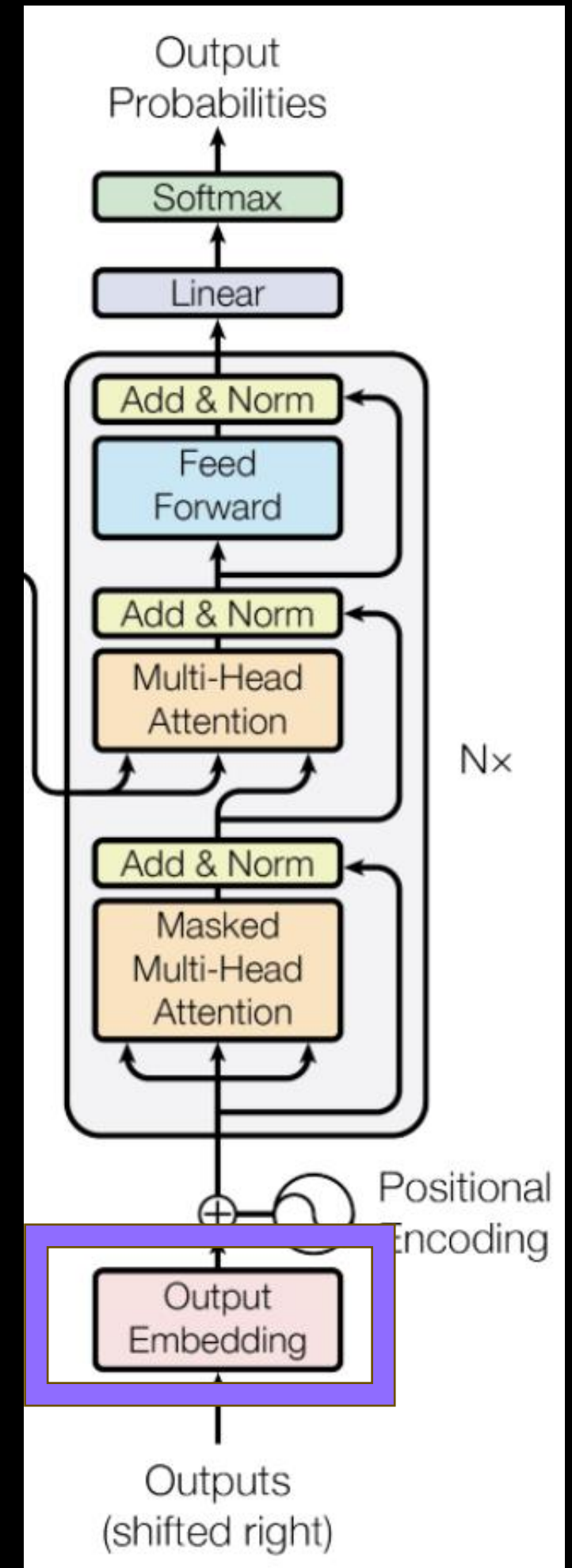
- Prep
 - Extraction
 - Chunking
 - Embedding
 - Vector DB loading





Challenge - Embedding

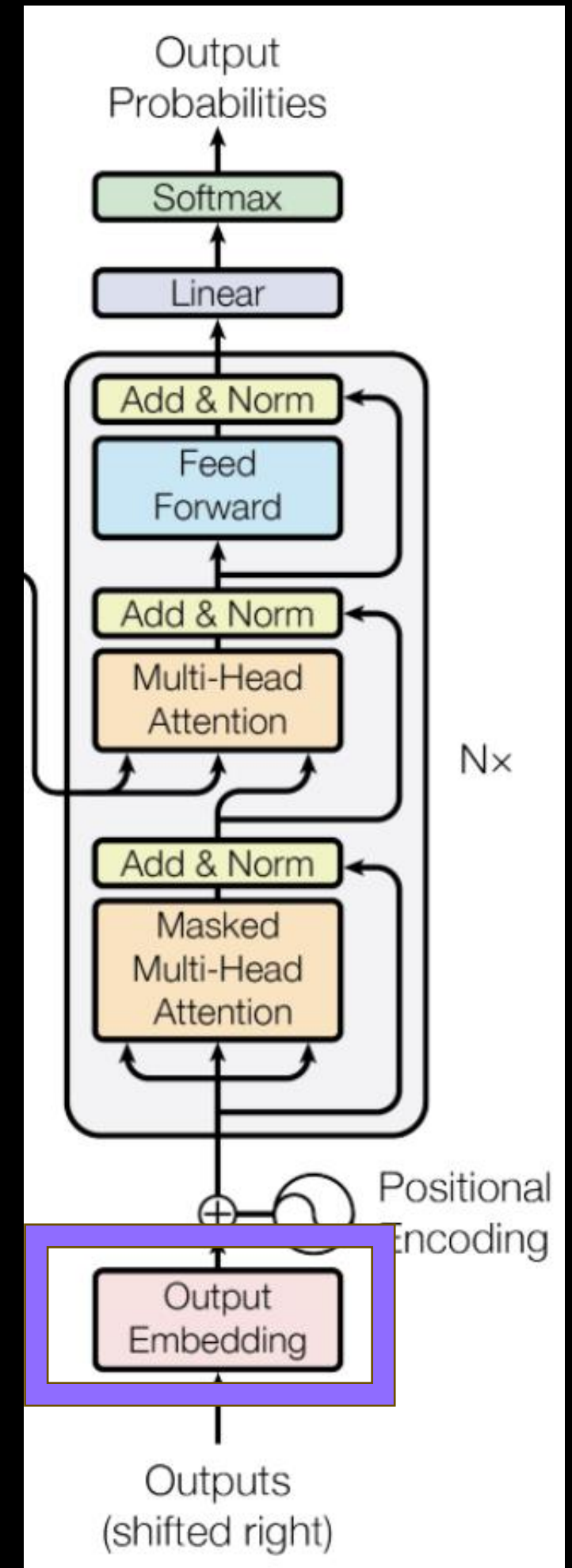
- 가용 Embedding method
 - LLM의 Embedding layers
 - 한국어로 Fine-tuning 된 모델이 많고
 - 가벼우나
 - Chunk를 Token단위로 임베딩한 후(shape=seq, dim)
 - Chunk를 하나의 Token으로 간주하여 계산(평균, shape=1,dim)하는 과정에서 의미 손실
 - Embedding model(Encoder-based feature extractor)
 - 모델이 문장을 입력받아 고정된 크기의 벡터로 연산하므로 성능은 좋으나
 - 무거우며 GPU 의존도가 높고(Parameters > 400M)
 - 한국어 특화 Embedding 모델 수가 많지 않음





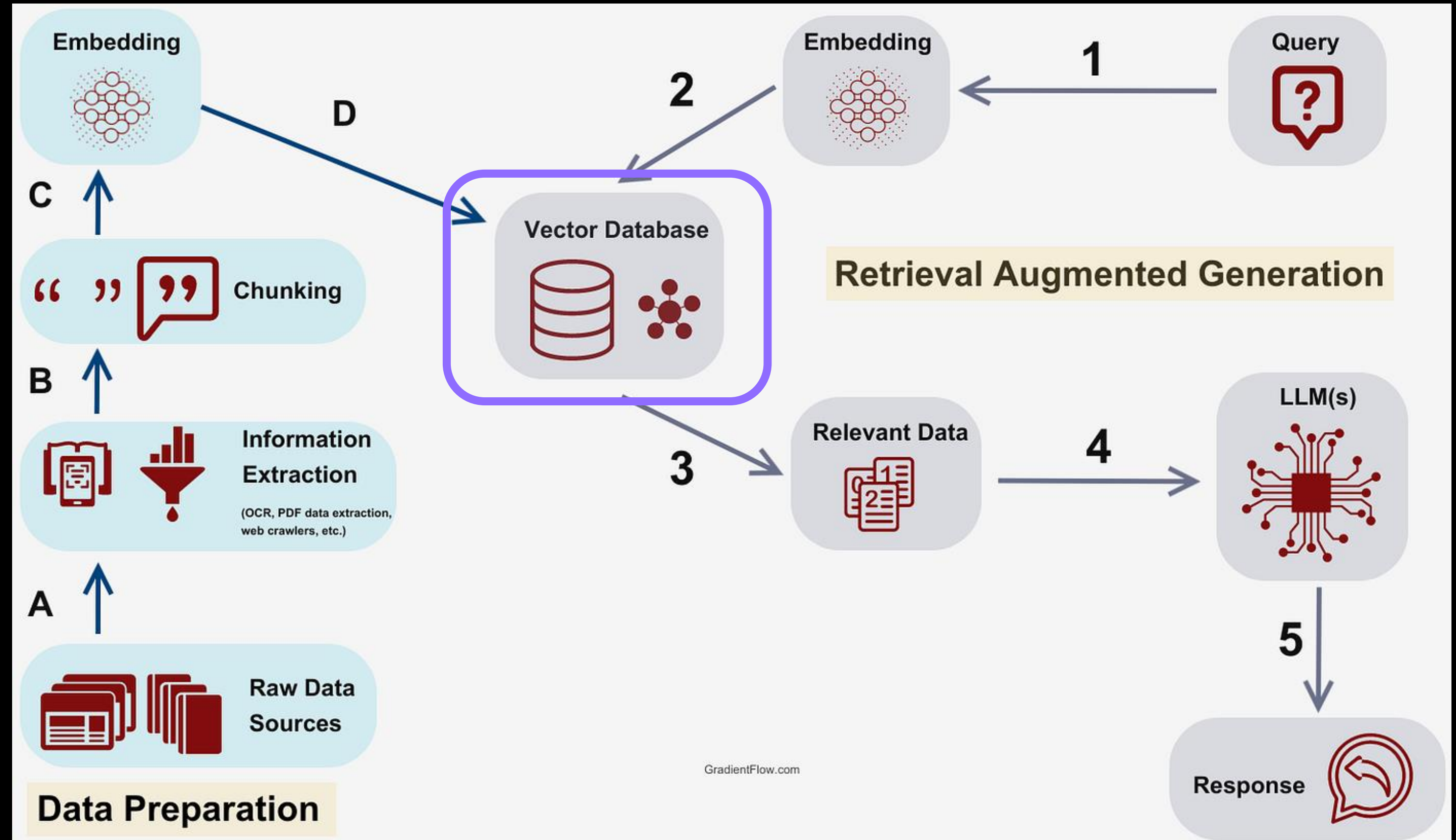
Tips - Embedding

- Embedding 모델의 경우 속도가 크게 중요하지 않음
 - Chunk embedding은 Back에서 연산되며, Query embedding과 async
 - Query embedding의 경우 GPU만 확보된다면, 1B 정도 모델의 경우 100ms 이하 처리 가능
 - SentenceTransformer 계열은 Quantization 불필요
- 다만, Domain-specific한 문서들을 정밀하게 읽어야 할 경우
 - Embedding model 별도 Fine-tuning 필요



Challenge – Retrieval

- RAG
 - Query embedding
 - Retrieval
 - Generation
 - Post-processing





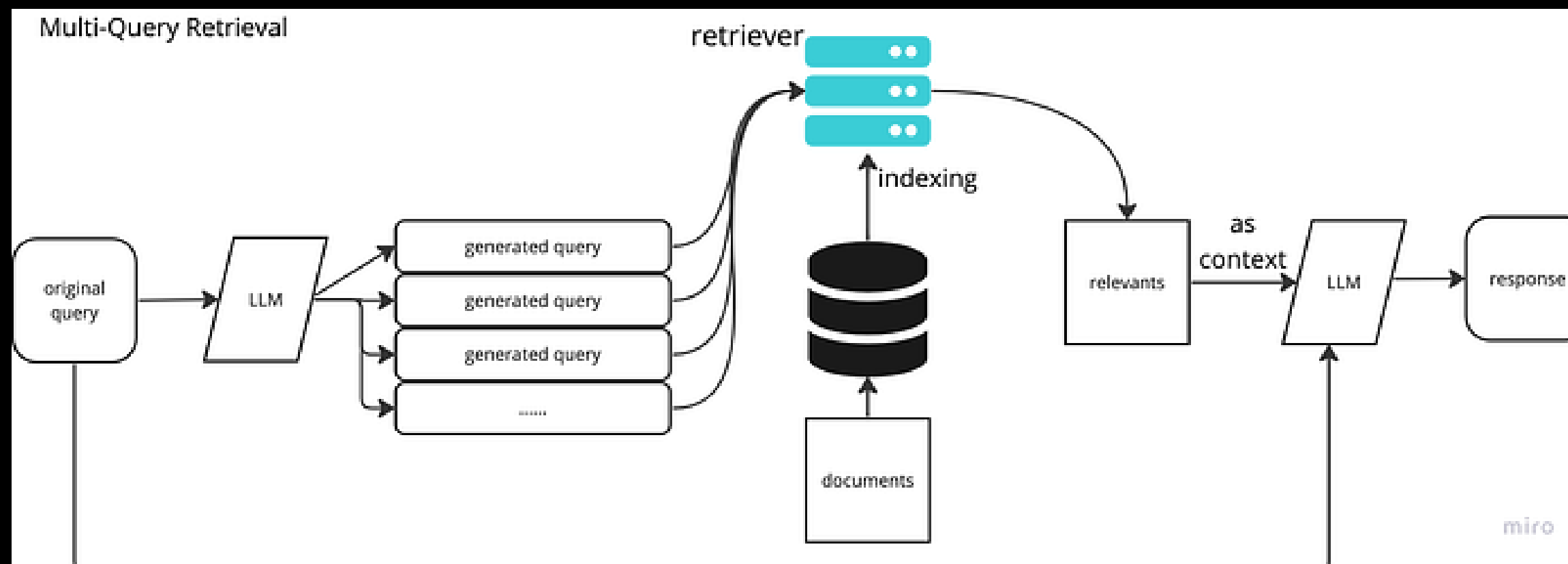
Challenge – Retrieval

- Query
 - 사용자의 요청
 - 두 가지 경로로 사용
 - Embedding 후 Vector DB에서 유사한 Chunk를 검색
 - 회수된 Chunk와 함께 LLM의 Prompt로 입력
 - Vector DB 구성 시 사용된 Embedding 모델과 동일하게 변환
 - 그러나 Chunk 검색 시 Query에 내포된 문제점 두 가지
 - 사용자의 질문을 온전하게 이해하지 못하는 경우
 - 사용자의 질문 그 자체와 유사한 Chunk를 검색하는 경우



Advanced RAG – Retrieval

- Multi-query
 - LLM으로 사용자의 Query를 증강하는 방법
 - Query를 서로 다른 N개의 문장으로 변환
 - 각 문장 별 검색된 Chunk들을 회수
 - 이들 중 중복을 제거
 - 경우에 따라 회수된 Chunk들에 대하여 Rerank 기법 적용 가능





Advanced RAG – Retrieval

- HyDE(Hypothetical Document Embedding)
 - 실제 우리가 Query와 유사도 중심으로 찾은 Chunk는 답변이 아닌 Query(질문)와 비슷한 문장
 - 이를 방지하기 위해 LLM을 활용하여 Query에 대한 **예비 답변**을 여러 개 생성
 - 각 답변과 유사한 Chunk를 검색한 후, 그 결과를 종합

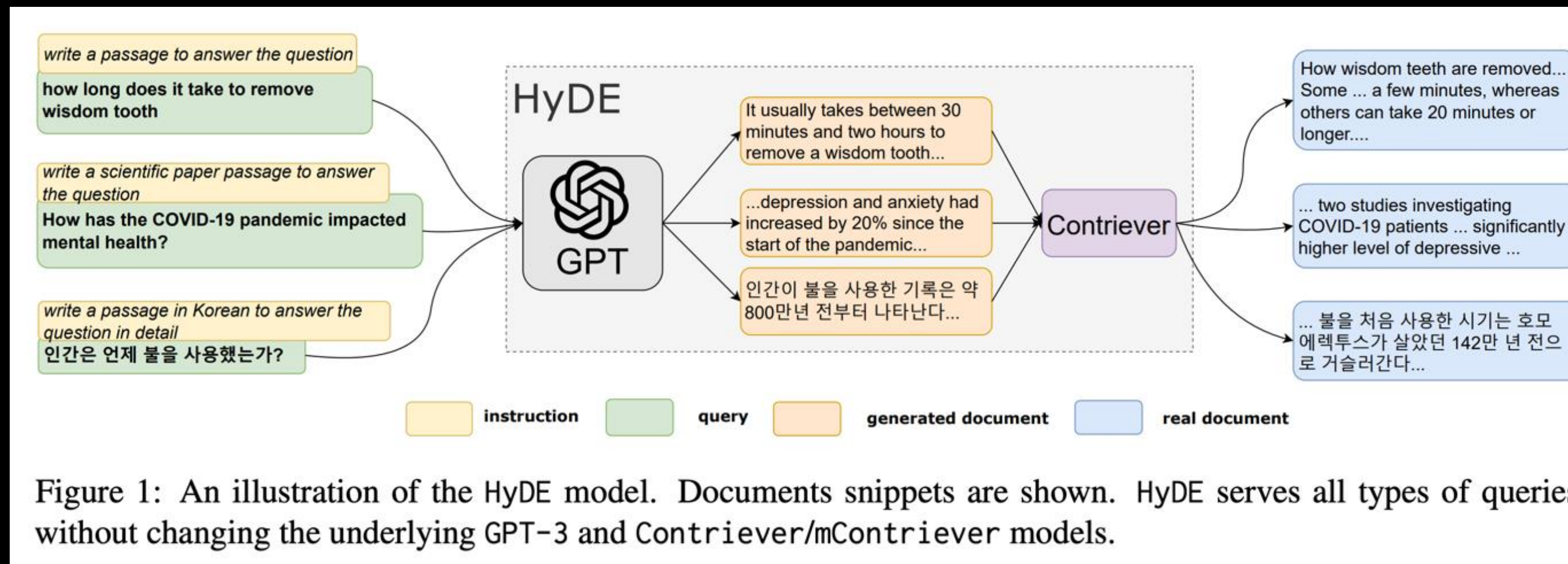
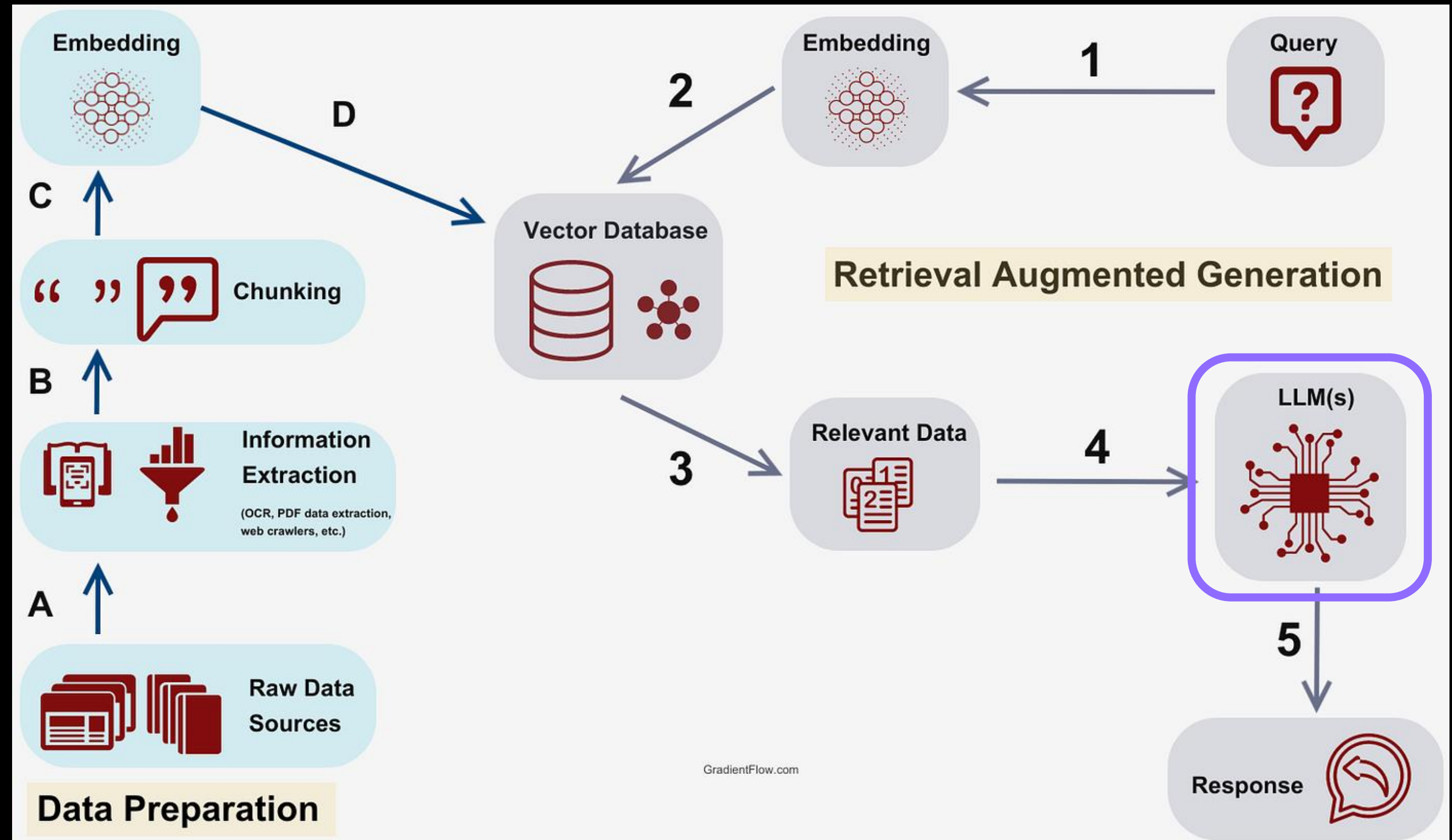


Figure 1: An illustration of the HyDE model. Documents snippets are shown. HyDE serves all types of queries without changing the underlying GPT-3 and Contriever/mContriever models.



Challenge – Generation

- RAG
 - Query embedding
 - Retrieval
 - **Generation**
 - Post-processing





Challenge - Generation

- Main LLM에서 Query와 Context를 기반으로 답변을 생성하는 과정
- UX에 가장 큰 가시적 영향을 미치는 부분이며, 아래 요소들이 이를 좌우(오픈 소스 기준)
 - Overall quality
 - 맞춤법, 어법을 잘 지키며, 태도, 역할 수행 측면에서 일관성을 지녀야 함
 - Contextual hallucination
 - Vector DB에서 Context가 제대로 회수되었다는 가정 하에, 이를 왜곡하지 않고 답변해야 함
 - Inference time
 - 최대 10-15s 이내 답변하는 것을 권장

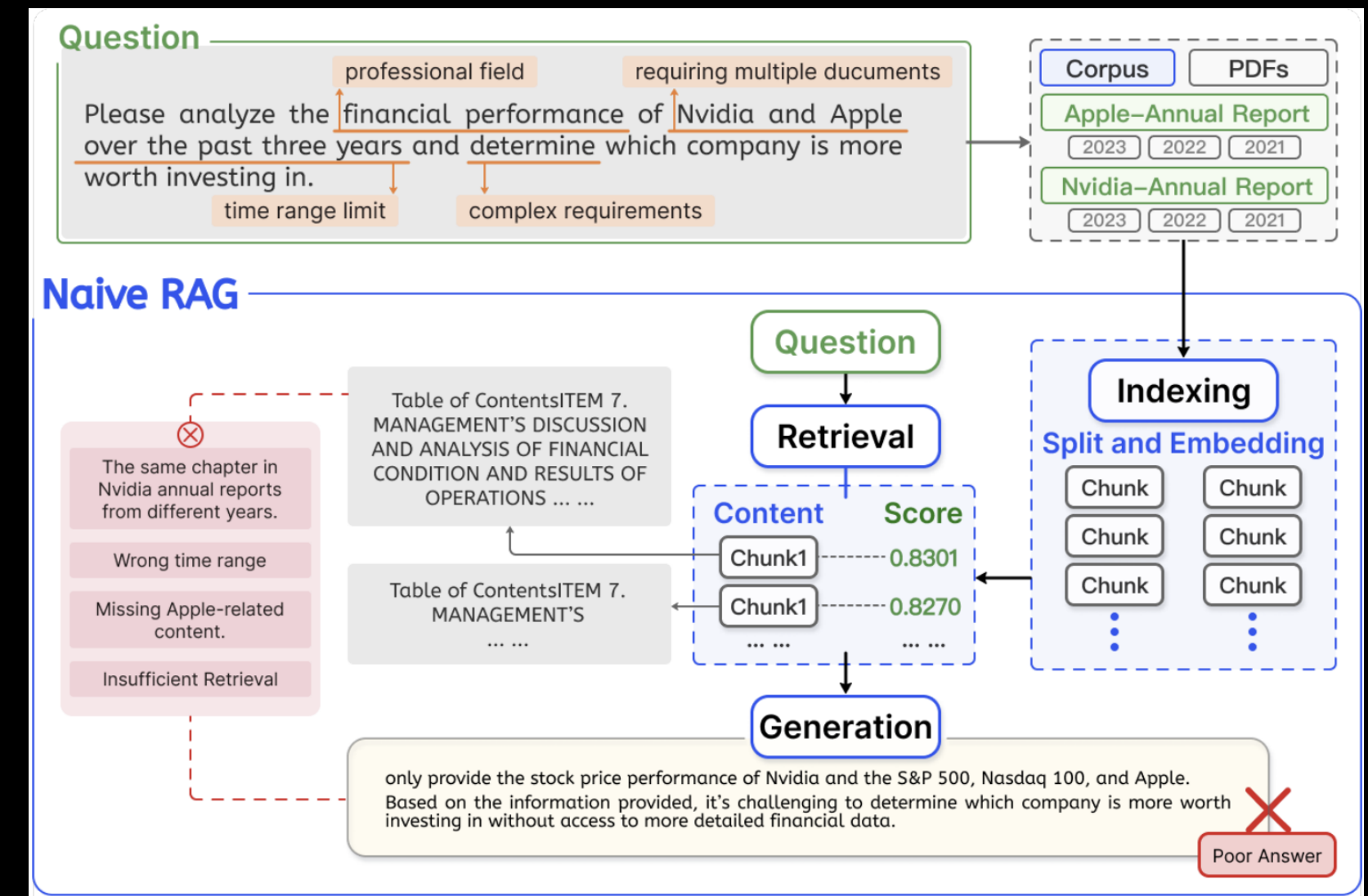


Tips - Generation

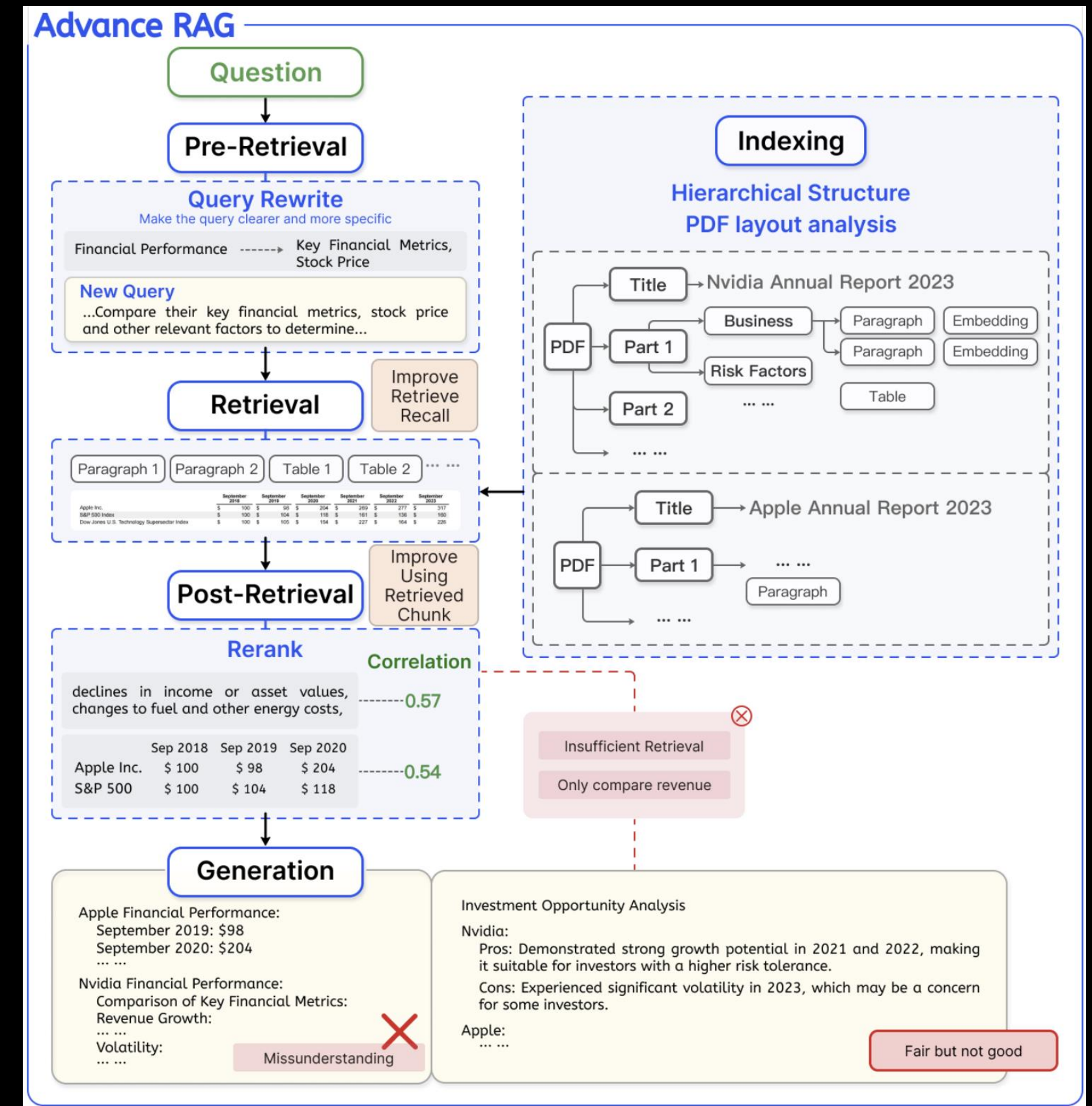
- RAG에서 LLM에 요구되는 역할은 크지 않음
 - Context와 Query를 자연스럽게 연결하여, 완결된 문장으로 작성하는 수준
 - 권장 사양(Rule of thumb)
 - LLaMA 3.1 이후 등장
 - 8B 이상의 파라미터 크기(Quantization 가능)
 - 가급적 한국어로 Pretraining된 모델 추천
- 다만 Inference time 최적화를 위해 다음 Application/tool 사용을 고려
 - Ollama
 - VLLM

2. Modular RAG

- Naïve RAG
 - 복잡한 Query를 이해하여 제대로 답변하지 못하는 한계
 - 복합 의문문, 지식과 상식을 모두 활용해야 하는 문제 등
 - 또는 지식 베이스(KB)가 다양한 문서로 구성될 경우
 - Query의 의미를 얕게 이해하거나, 불필요한 정보를 함께 활용하여 Hallucination 발생



- Advanced RAG
 - Naïve RAG의 문제를 해결하기 위하여 다양한 기법 사용
 - Self-RAG, Adaptive RAG, HyDE, Multi-query, Reranking, Hybrid search
 - 사용자 요구와 응용 시나리오를 다루기 위한 필요성 대두
 - 텍스트, 표, 그래프 등 이질적인 데이터를 통합
 - 시스템 투명성(Explainability)
 - 제어 및 유지보수
 - 비순차적 워크플로우 제어 및 스케줄링





- Modular design
 - 시스템을 더 작은 단위인 모듈로 분할하여 설계하는 방법론
 - 각 모듈이 서로 독립적으로 구성되며, 다른 모듈을 고려하지 않고도 생성, 수정, 대체될 수 있도록 설계
 - 소프트웨어 시스템 뿐만 아니라 기계 및 공정 디자인, 사무실, 주택 건설 시에도 적용



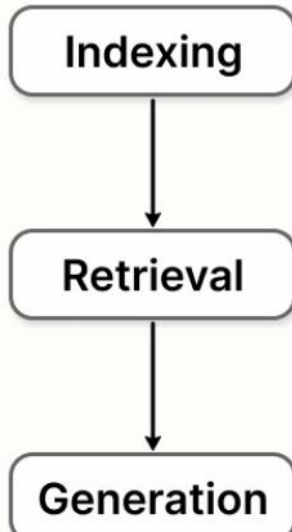
Modular RAG

- Modular RAG
 - 논문 [Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks, 2024]에서 제안
 - RAG 시스템을 독립적인 모듈과 특화된 연산자로 분해하여
 - 레고 블록을 조립하듯 유연하게 구성 변경이 가능한 프레임워크를 구축하는 것이 목표
 - RAG 및 LLM과 IR 관련 연구들이 넘쳐나는 상황 속에서, 시스템 구성의 유연성을 높여 확장성 개선

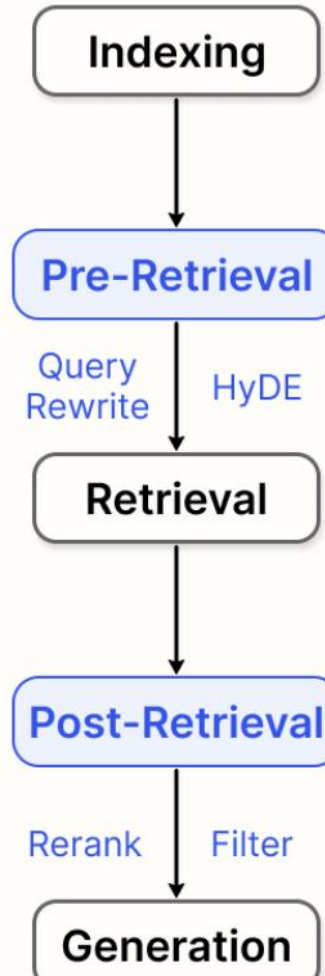


Modular RAG

Naive RAG



Advanced RAG



Legend

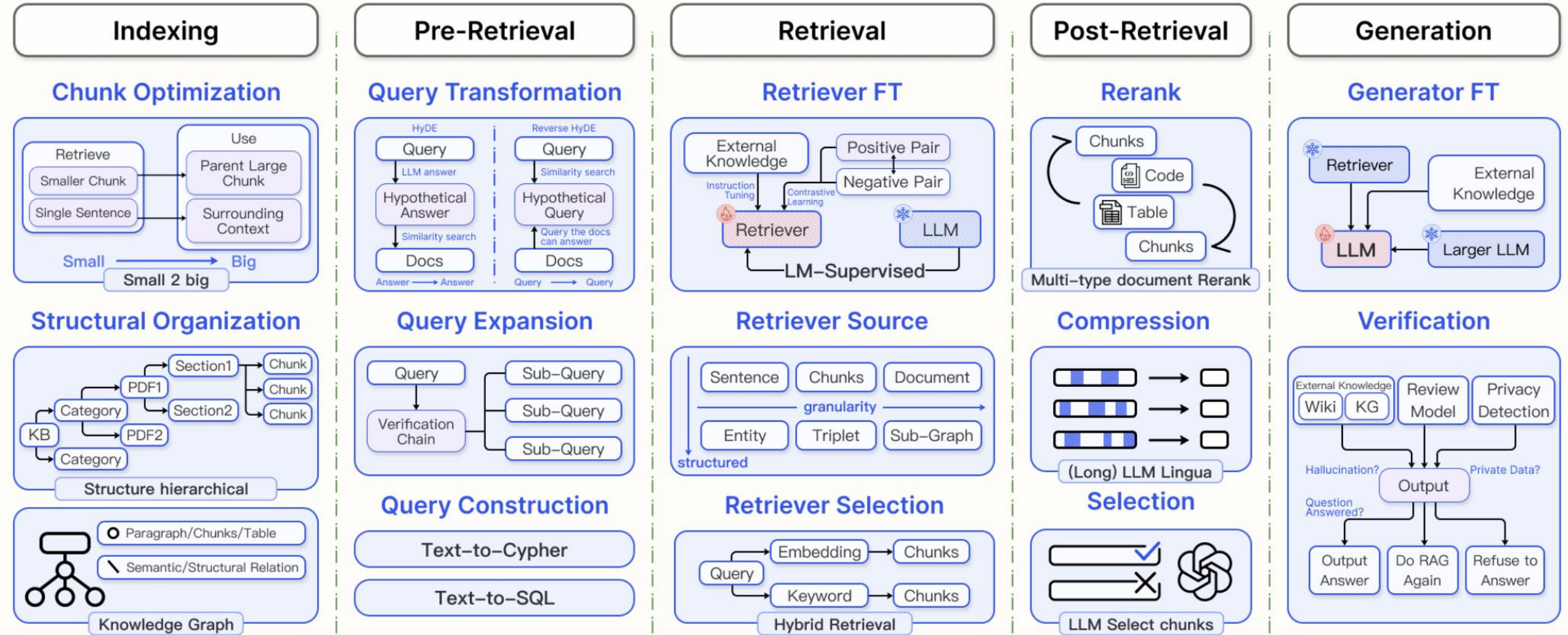


Naive

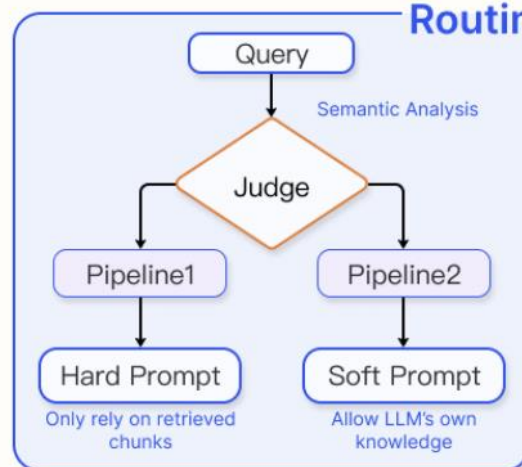
Advance

Modular

Modular RAG

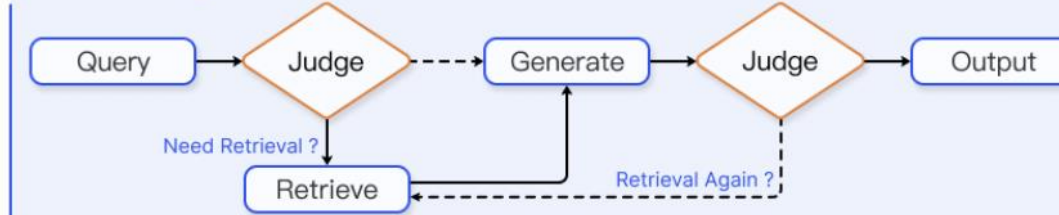


Routing

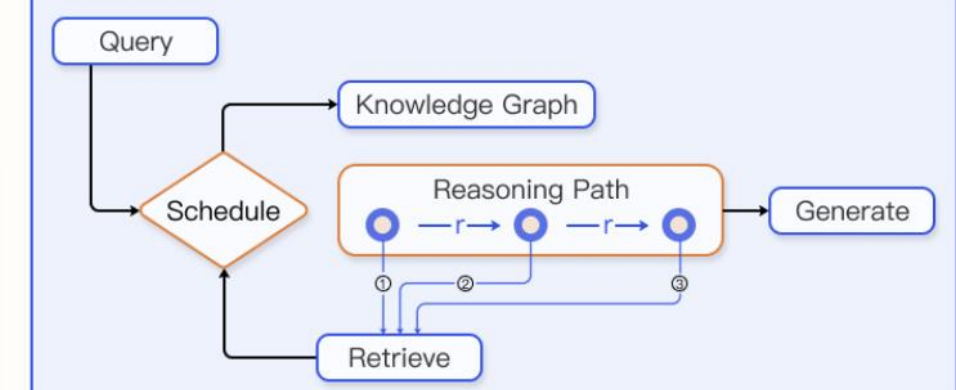


Orchestration

Scheduling



Knowledge Guide



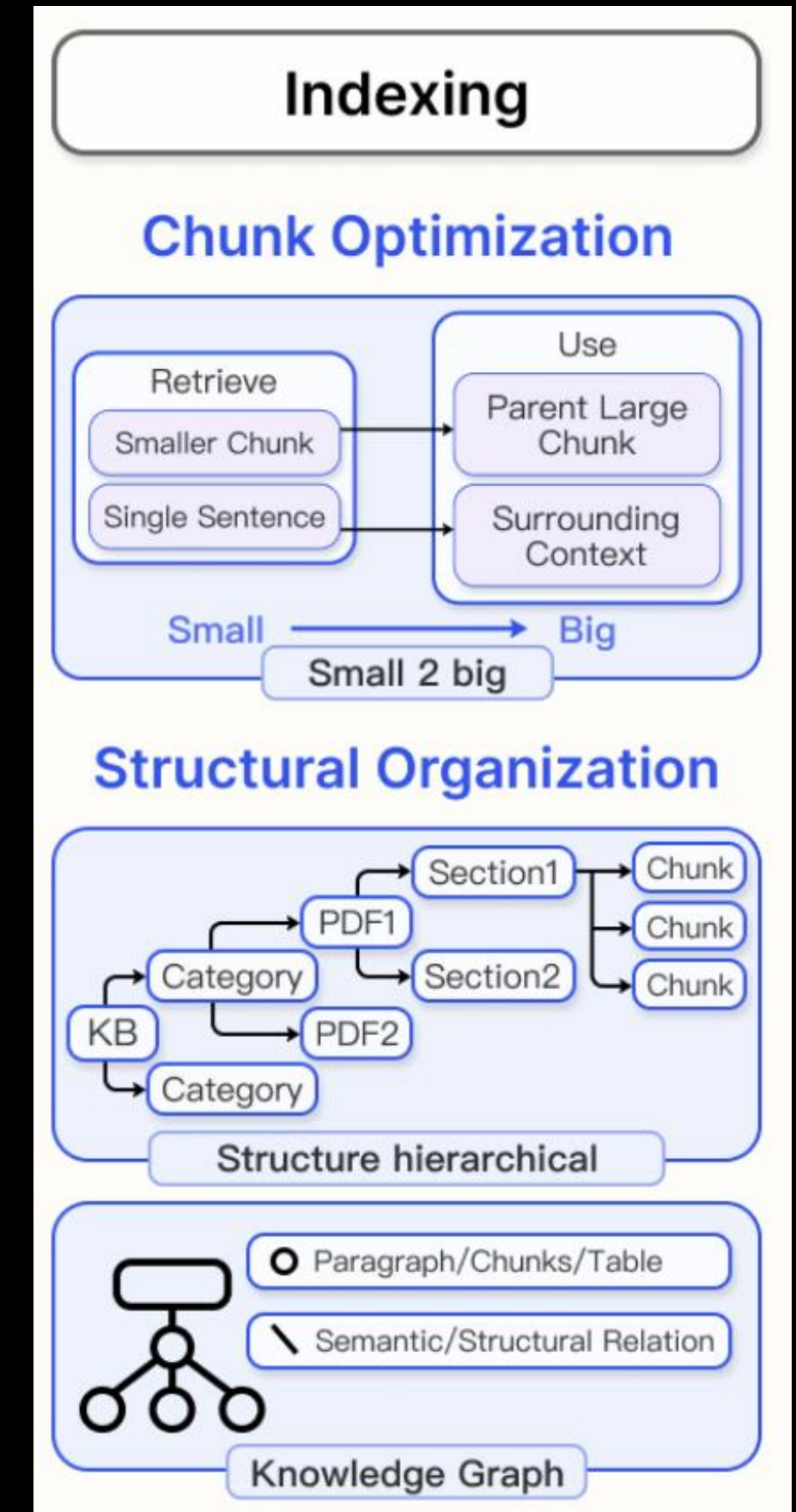


Modular RAG

- 논문에서는 RAG 시스템의 구성 요소를 분석하여, 3계층 모듈 구조로 재구성
 - L1
 - RAG 시스템의 주요 단계인 최상위 계층
 - Indexing(문서 색인), Pre-retrieval(검색 전 처리), Retrieval(문헌 검색), Post-retrieval(검색 후 처리), Generation(LLM 기반 답변 생성), **Orchestration**(워크플로우 제어)
 - L2
 - 각 모듈의 하위 모듈(Sub-module)
 - L1 내의 기능적 단계들
 - L3
 - 실제 연산 및 작업을 수행하는 최소 단위로, 연산자 레벨(Operator)이라고도 정의

Modules

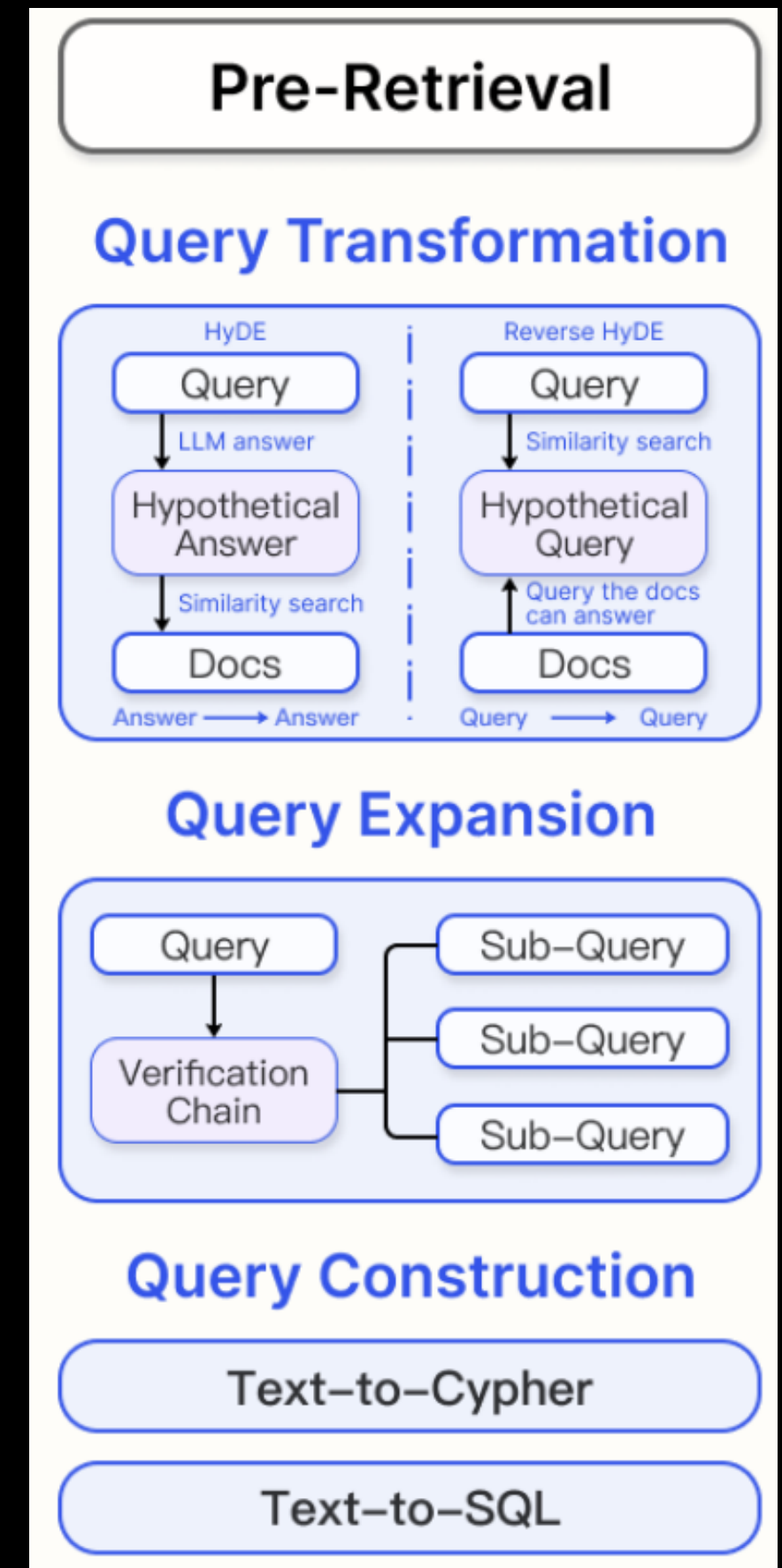
- Indexing module의 L2 & L3
 - 데이터 전처리
 - 정제, 토큰화, 정규화
 - 임베딩 생성
 - TF-IDF, BM25, Sentence transformer
 - Index 생성
 - Inverted idx 구축, Index 계층화, Index 최적화 알고리즘 구축
 - 데이터 저장 및 관리
 - Vector DB 구축, DB caching, DB 구조화, 업데이트





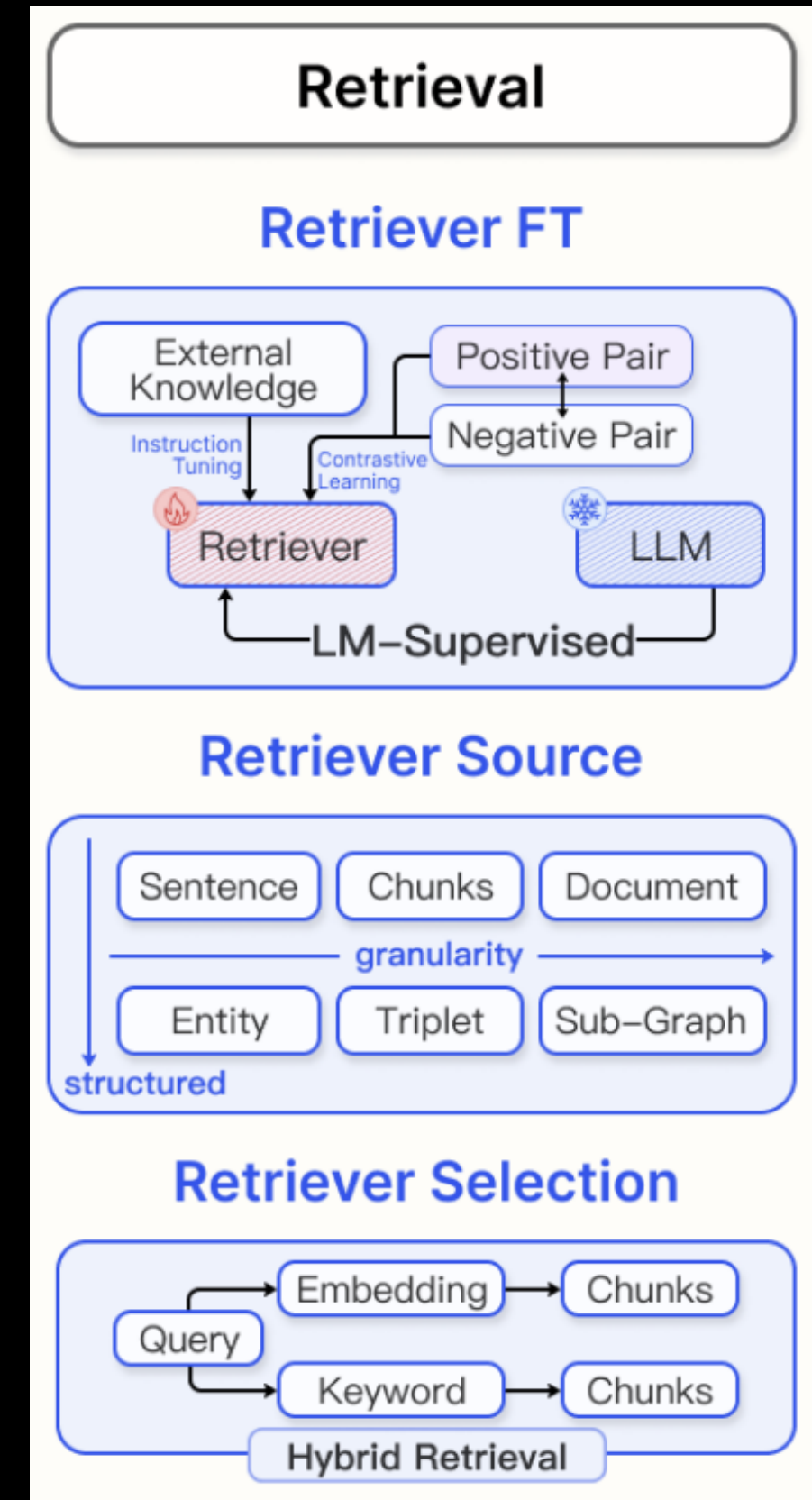
Modules

- Pre-retrieval module의 L2 & L3
 - Query 분석
 - 의도 분류, 키워드 추출 알고리즘, 구문 트리 기반 분석
 - Query 재작성
 - LLM 기반 Query paraphrasing, HyDE, 구조 재조정 및 유의어 대체기
 - Query 확장(Multi-querying)
 - 동의어 확장 모듈, Context 기반 Query 확장
 - 문맥 보강
 - 웹 기반 외부 지식 연동, 도메인 정보 삽입, LLM 기반 배경 설명 생성

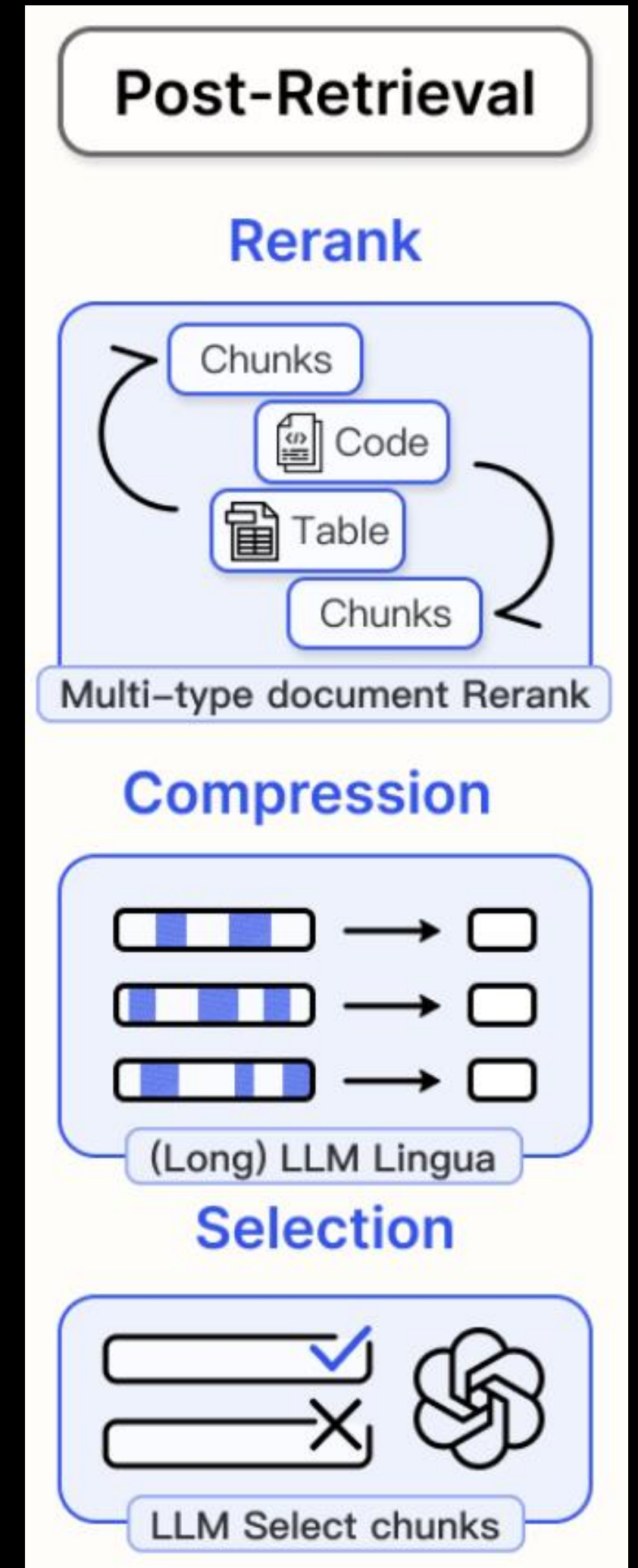


Modules

- Retrieval module의 L2 & L3
 - 기본 검색
 - BM25 score 계산기, Inverted idx 검색, 키워드 매칭 모듈
 - 유사도 기반 검색
 - Cosine, L2 distance, MMR, KNN
 - 다중 검색 모듈
 - 검색 전략 선택기, 병렬 검색 실행 모듈, 결과 병합 모듈
 - 결과 집계
 - 결과 필터링 연산자, 결과 rank 조정기, 중복 제거 모듈

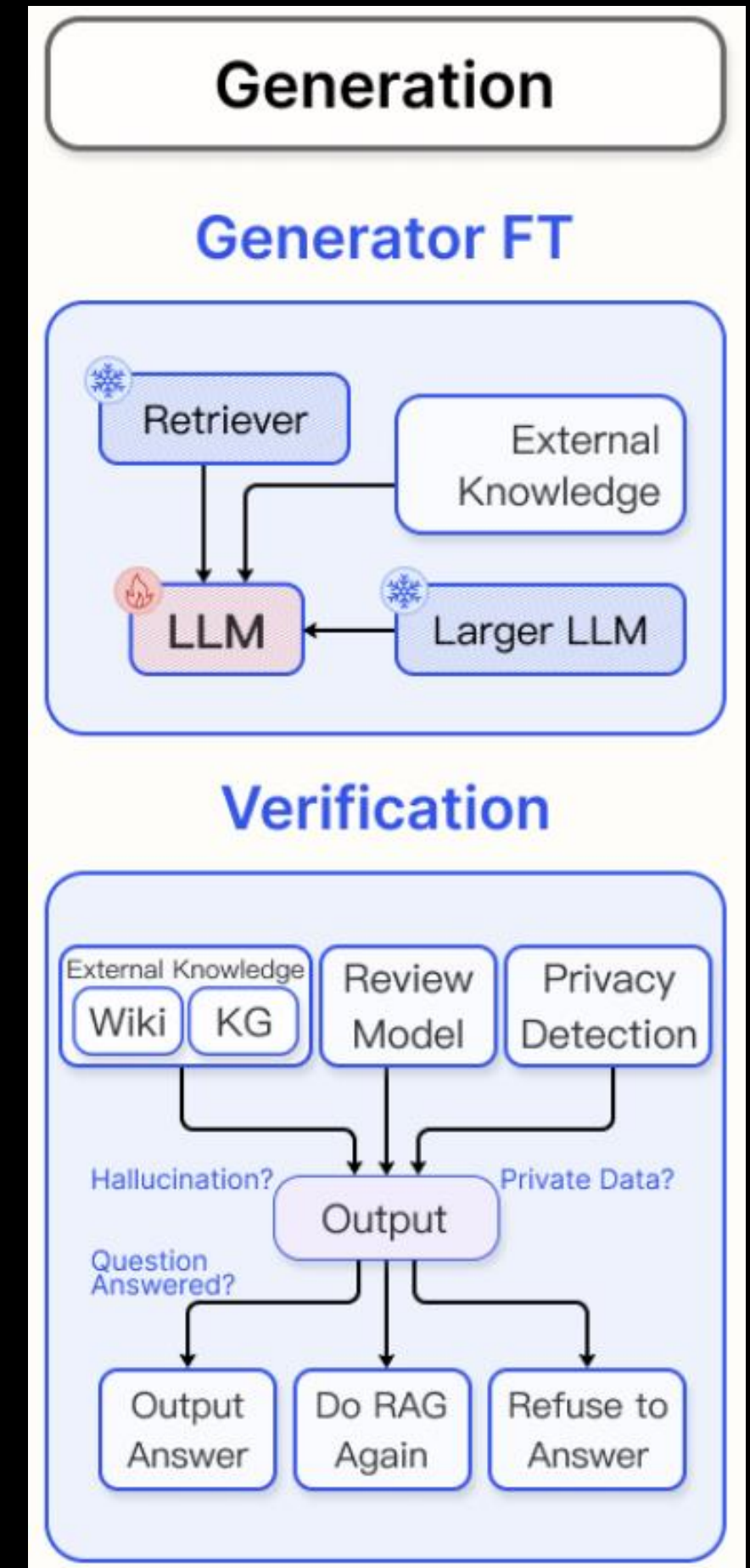


- Post-retrieval module의 L2 & L3
 - 결과 필터링
 - 노이즈 제거 필터, 유사도 Threshold 필터링, 스팸 검출 모듈
 - 결과 재정렬
 - Reranking 알고리즘, Alignment 함수, 사용자 피드백 반영 모듈
 - 결과 요약
 - 추출적 요약 모듈(Parameterless), 생성적 요약 모듈, 문단 요약 알고리즘
 - 결과 평가
 - Context recall/precision 평가 모듈, 사용자 만족도 피드백 분석 모듈



Modules

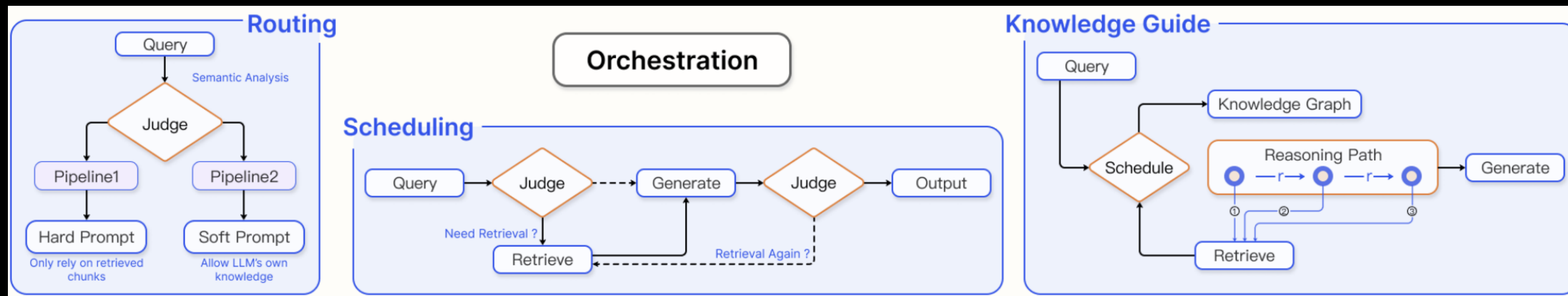
- Generation module의 L2 & L3
 - 입력 설계 및 제어
 - Prompt template 생성, LLM API 호출 모듈, 응답 캐시 관리 모듈
 - 후처리
 - 문법 교정 모듈, 텍스트 정제 함수, 포맷(Markdown 등) 변환 도구
- Feedback Loop
 - 결과 평가 자동화 모듈, 재생성 Trigger





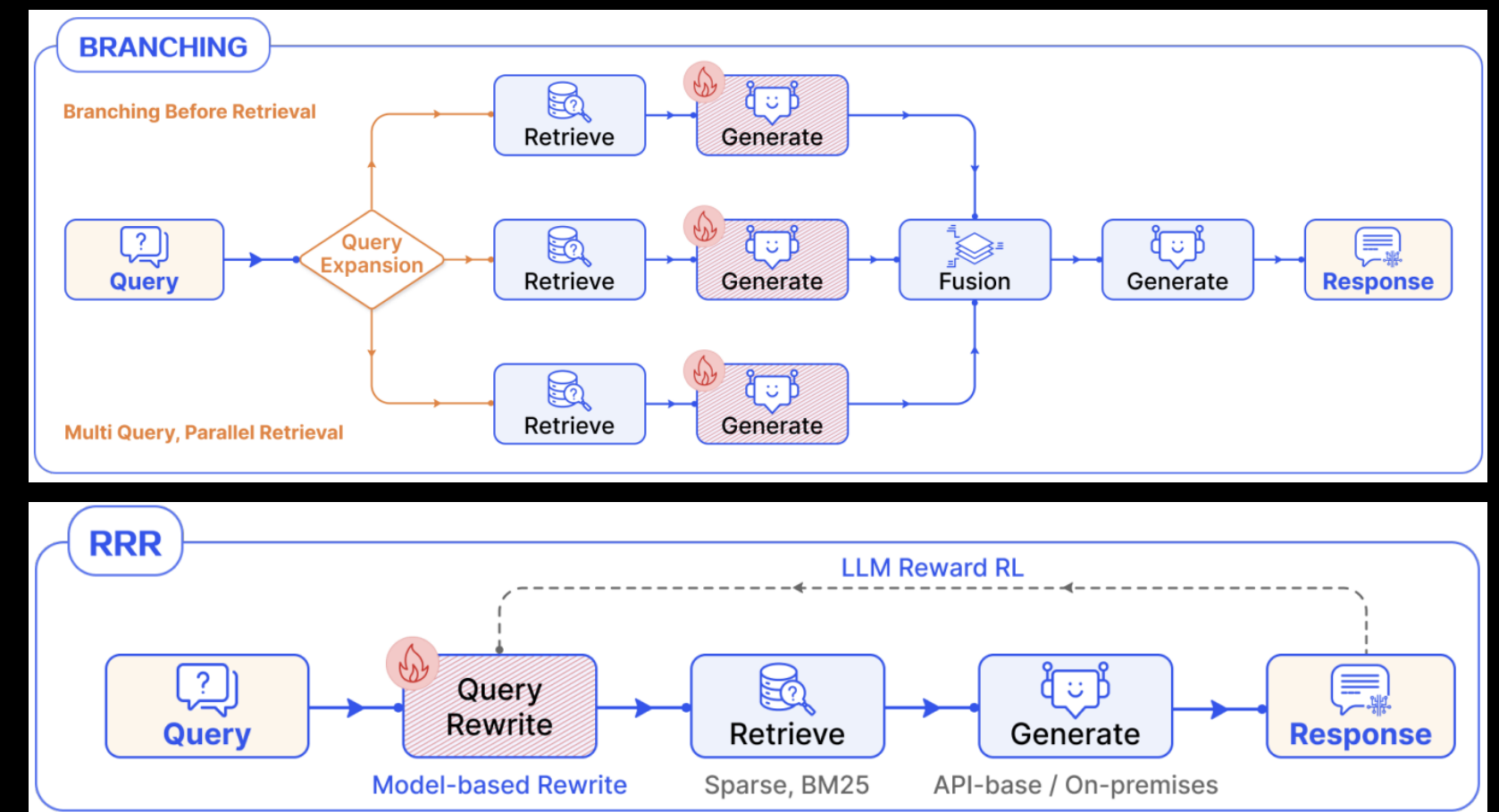
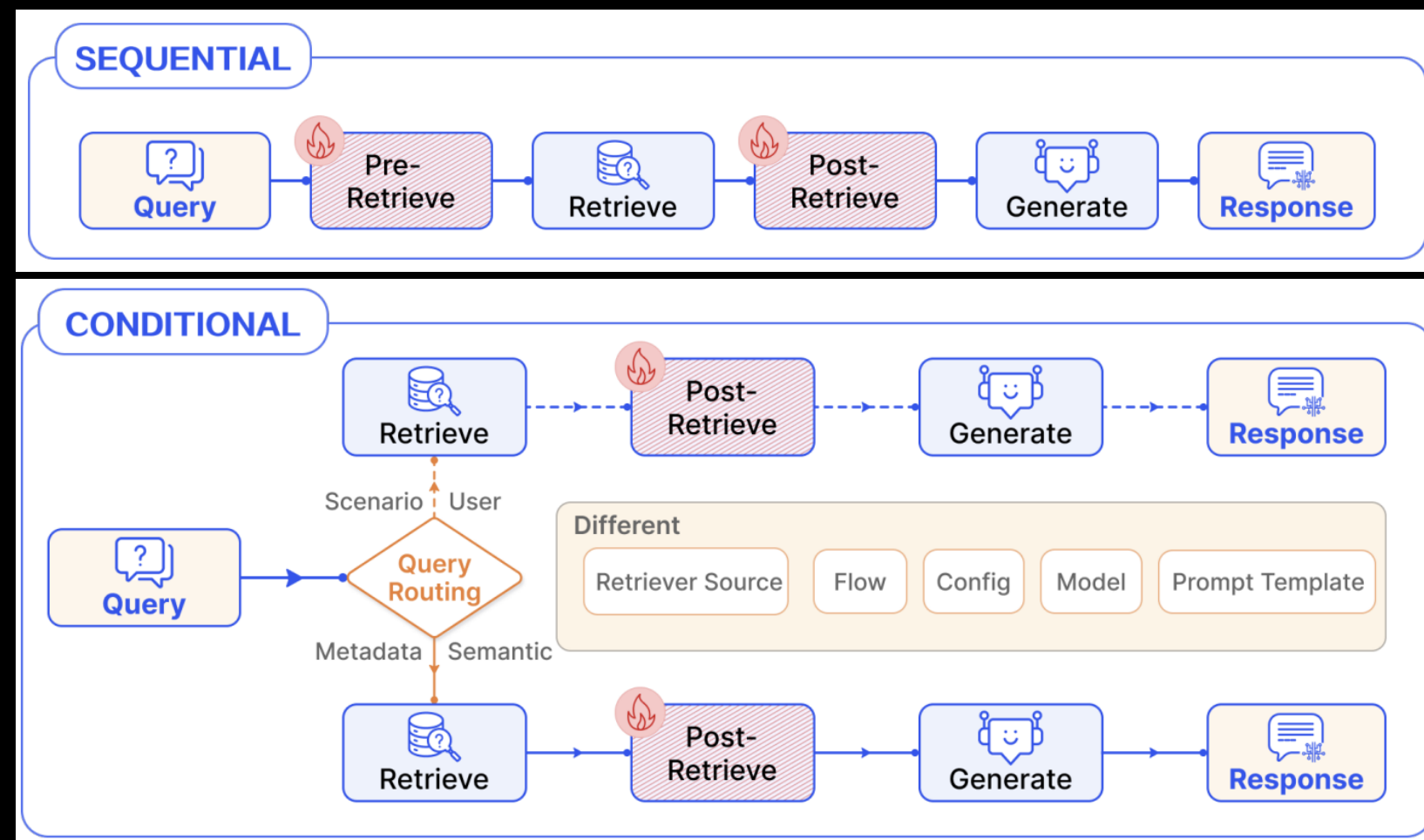
Modules

- Orchestration module
 - Prompt, context 및 결과들에 대한 흐름과 이에 대한 제어, 동적인 프로세싱을 담당
 - Iterative retrieval
 - 질문이 충분히 답변되지 않았다고 판단되면 이전 응답을 바탕으로 새로운 질의를 생성하여 다시 검색
- Adaptive RAG
 - 충분한 정보를 모을 때까지 루프를 돌거나 웹 검색 등 기타 도구를 활용
- 이러한 동적인 제어를 정의 및 관리하기 위하여 RAG 시스템에서 관찰되는 흐름 패턴을 분류



Modules

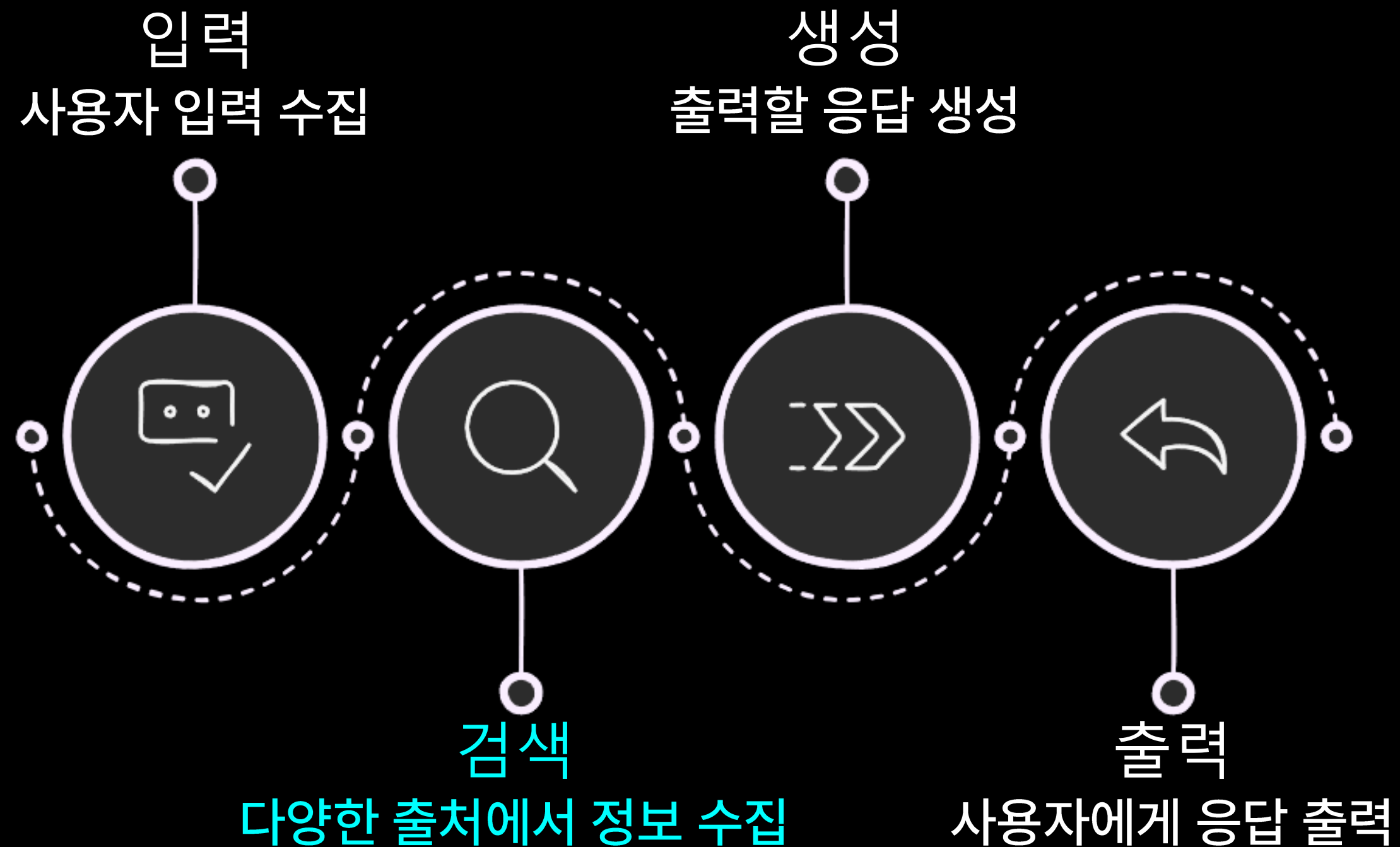
- RAG 시스템에서 관찰되는 흐름 패턴의 분류
 - 선형(Sequential) 패턴
 - 조건부(Conditional) 패턴: Router 함수를 통해 조건에 따라 다른 모듈 경로를 선택 및 실행
 - 분기(Branching) 패턴: 다양성 향상 및 효율 증진을 위하여 병렬 수행 및 병합(Fan-out/fan-in)
 - 반복(Loop) 패턴: 조건을 만족할 때까지 특정 작업을 반복





RAG 시스템

- 이번 실습에서는 모듈을 **입력, 검색, 생성, 출력** 네 가지 구성 요소로 구분
- 특히 검색 과정을 세분화하여 모듈화 시스템으로 리팩토링 할 예정





검색 과정 고도화

- 검색 과정을 고도화 하는 방법
 - RAG: Vector Embedding 및 유사도 검색 알고리즘의 고도화
 - Rerank: 검색 결과를 재정렬하여 연관성이 높은 정보를 우선적으로 배치하는 후처리 기법
 - Adaptive RAG: 사용자의 질문에 따라 적합한 기법을 선택해서 활용하는 기법
- 위 고도화 기법을 서로 독립적으로 모듈화 해서 활용할 수 있을까?



검색 모듈의 구성

- 검색 모듈의 입력과 출력은 직관적으로 규정할 수 있음
 - 입력: 사용자의 입력, (원하는 문서 개수)
 - 출력: 검색된 문서

입력
사용자의 입력, 원하는 문서의 수

검색 모듈

출력
검색된 문서



검색 모듈의 구성

- 검색 모듈의 입력과 출력은 L2 & L3직관적으로 규정할 수 있음
 - 입력: 사용자의 입력, (원하는 문서 개수)
 - 출력: 검색된 문서
- 그렇다면, 앞서 학습한 고도화 기법을 어떻게 모듈화하여 적용할 수 있을까?

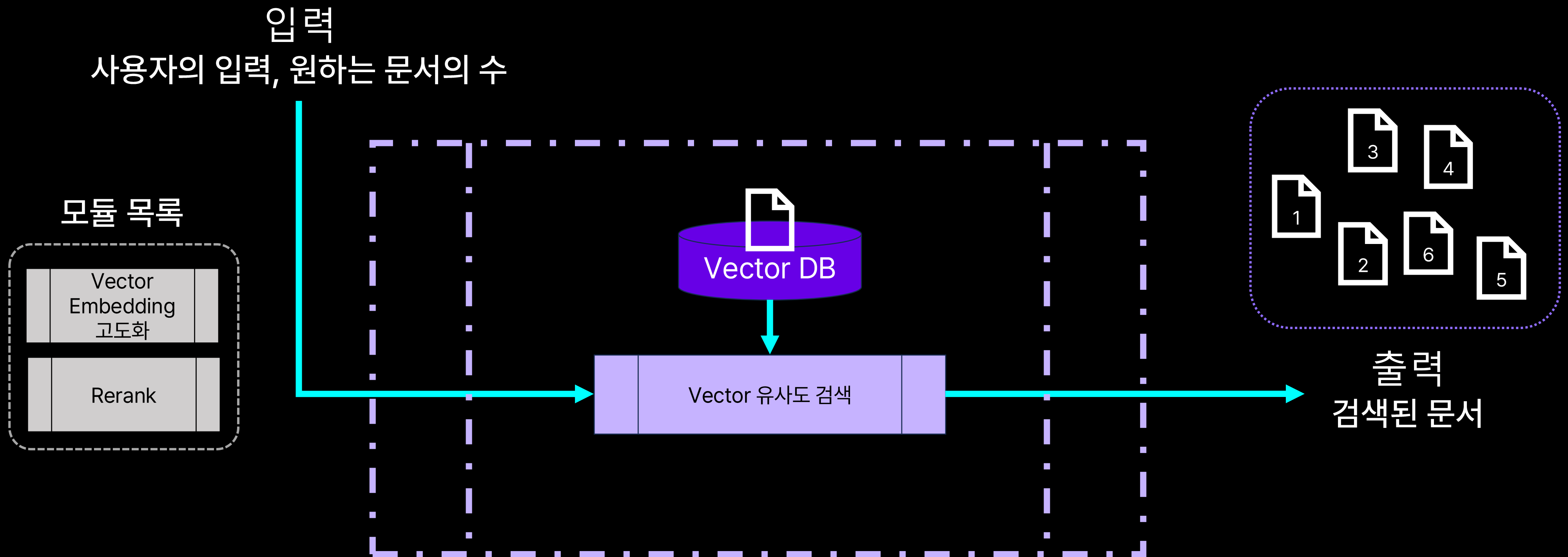
입력
사용자의 입력, 원하는 문서의 수



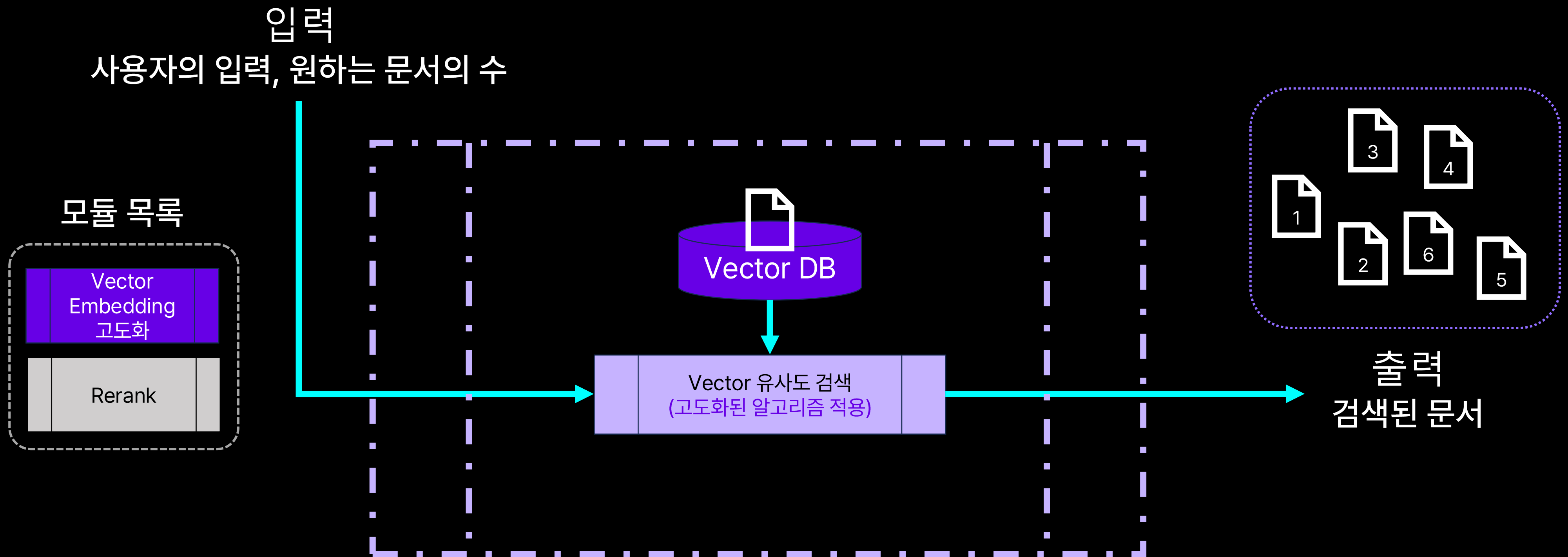
출력
검색된 문서

간단한 검색 모듈

- 입력과 출력 규정을 지키는 간단한 검색 모듈은 아래와 같음:



- Vector Embedding 고도화: Vector 유사도 검색 과정에서 고도화된 알고리즘을 적용
 - 적용 전 후 입력과 출력 형식이 변하지 않는 것을 확인할 수 있음

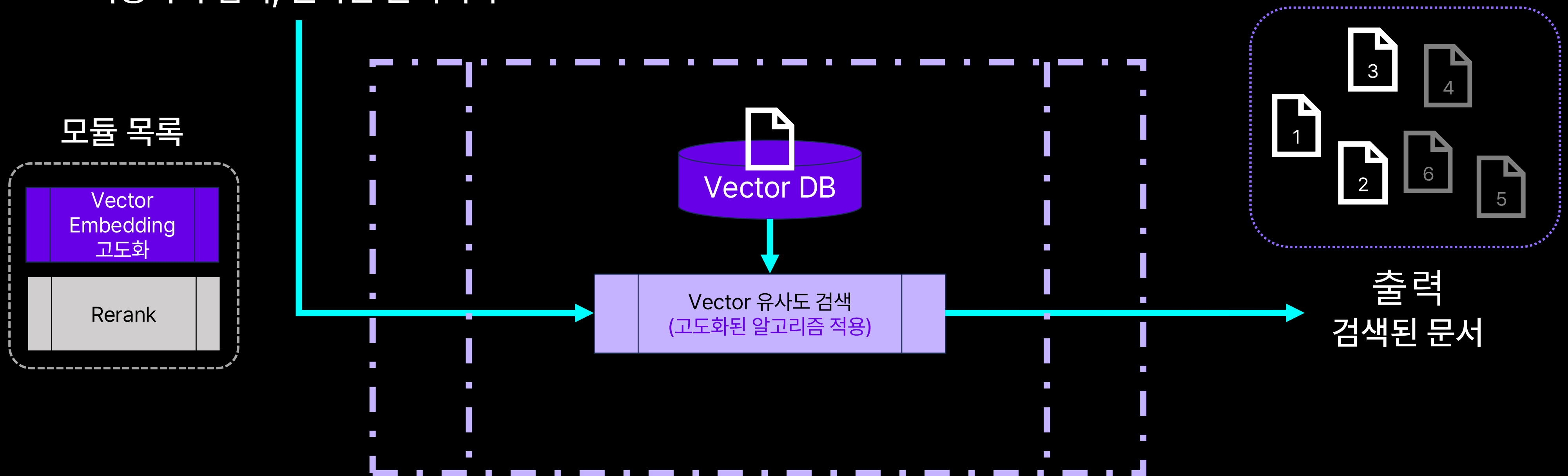


모듈 교체

- Vector Embedding 고도화: Vector 유사도 검색 과정에서 고도화된 알고리즘을 적용
 - 적용 전 후 입력과 출력 형식이 변하지 않는 것을 확인할 수 있음
 - 문서가 검색되는 것은 변하지 않지만, 검색되는 문서의 종류, 양, 퀄리티 등은 모두 변할 수 있음

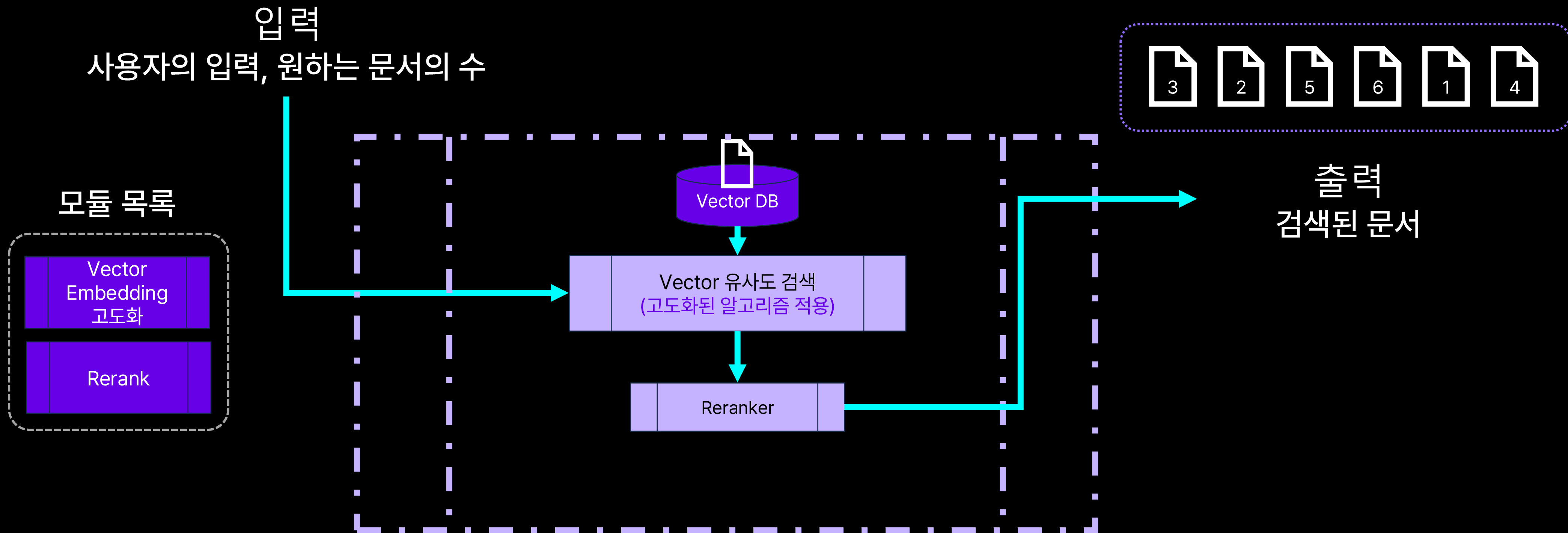
입력

사용자의 입력, 원하는 문서의 수



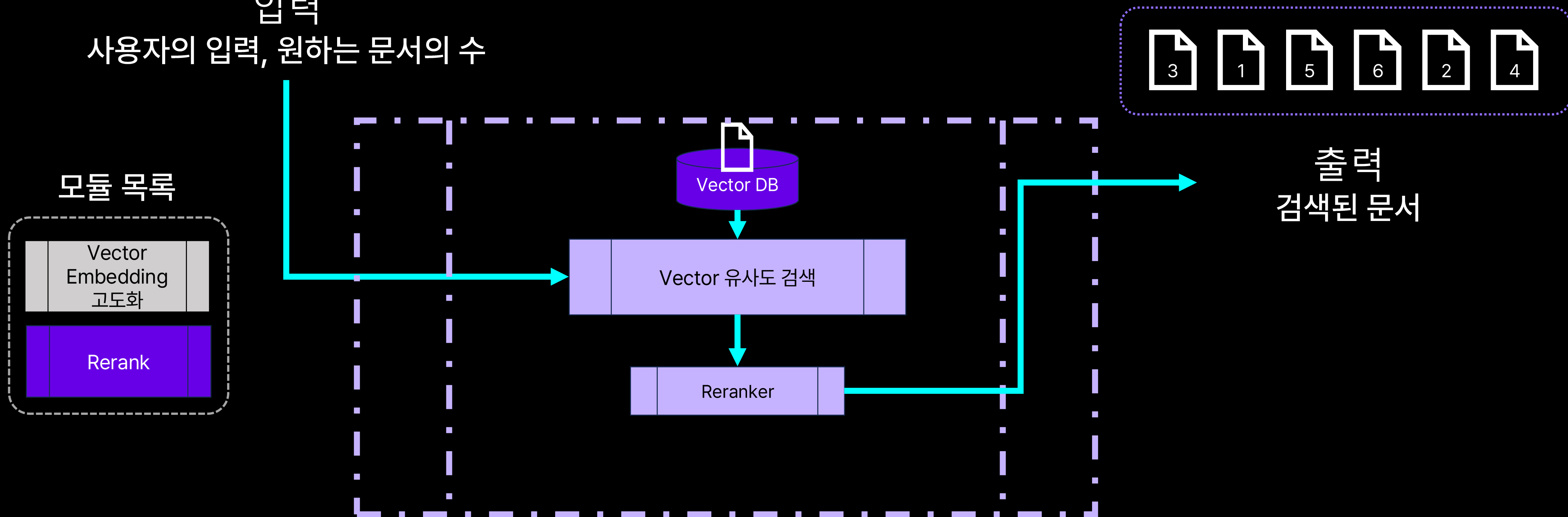
모듈 교체

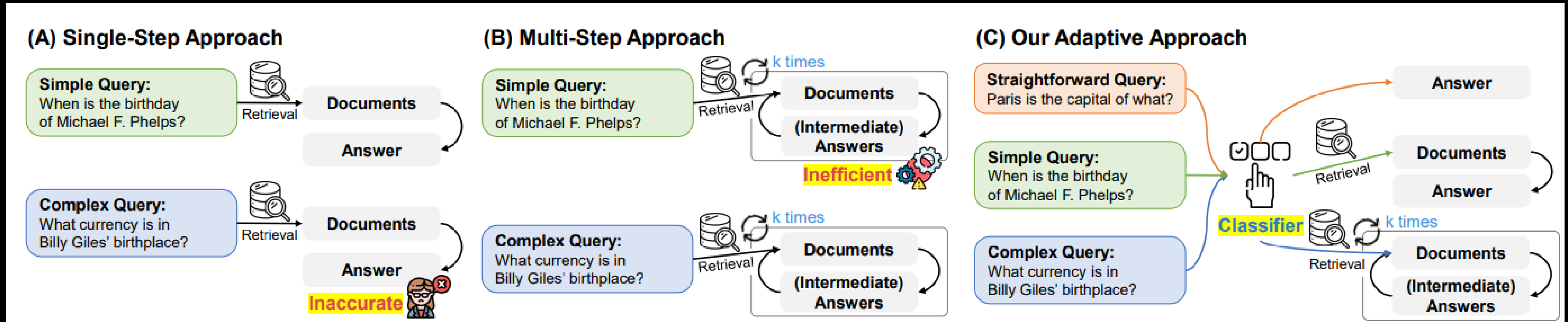
- Rerank: 검색 결과를 재정렬하여 연관성이 높은 정보를 우선적으로 배치
 - 적용 전 후 입력과 출력 형식이 변하지 않는 것을 확인할 수 있음



- 인력

사용자의 입력, 원하는 문서의 수



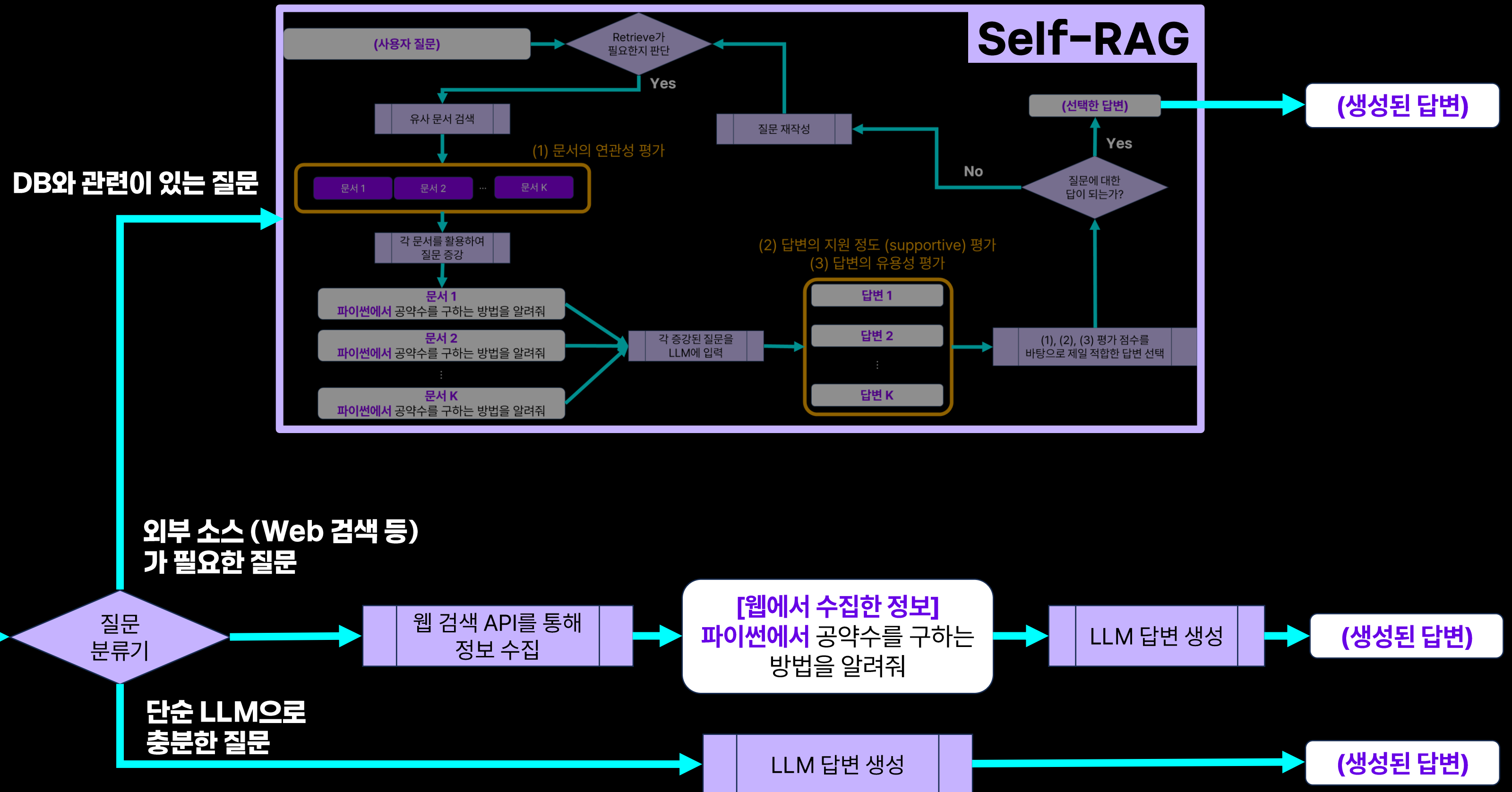


• 별도의 분류기를 활용하여 질문의 종류에 따라

- (1) 단순 답변
 - (2) 단일 검색을 통한 RAG 답변
 - (3) 여러 번의 검색 및 추론을 통한 RAG 답변을 생성하는 적응형 RAG 시스템
- 입력된 질문(Query)에 적합한 형태로 적응(Adapt)하여 답변하는 시스템



Adaptive RAG



Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화 되었다고 가정

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



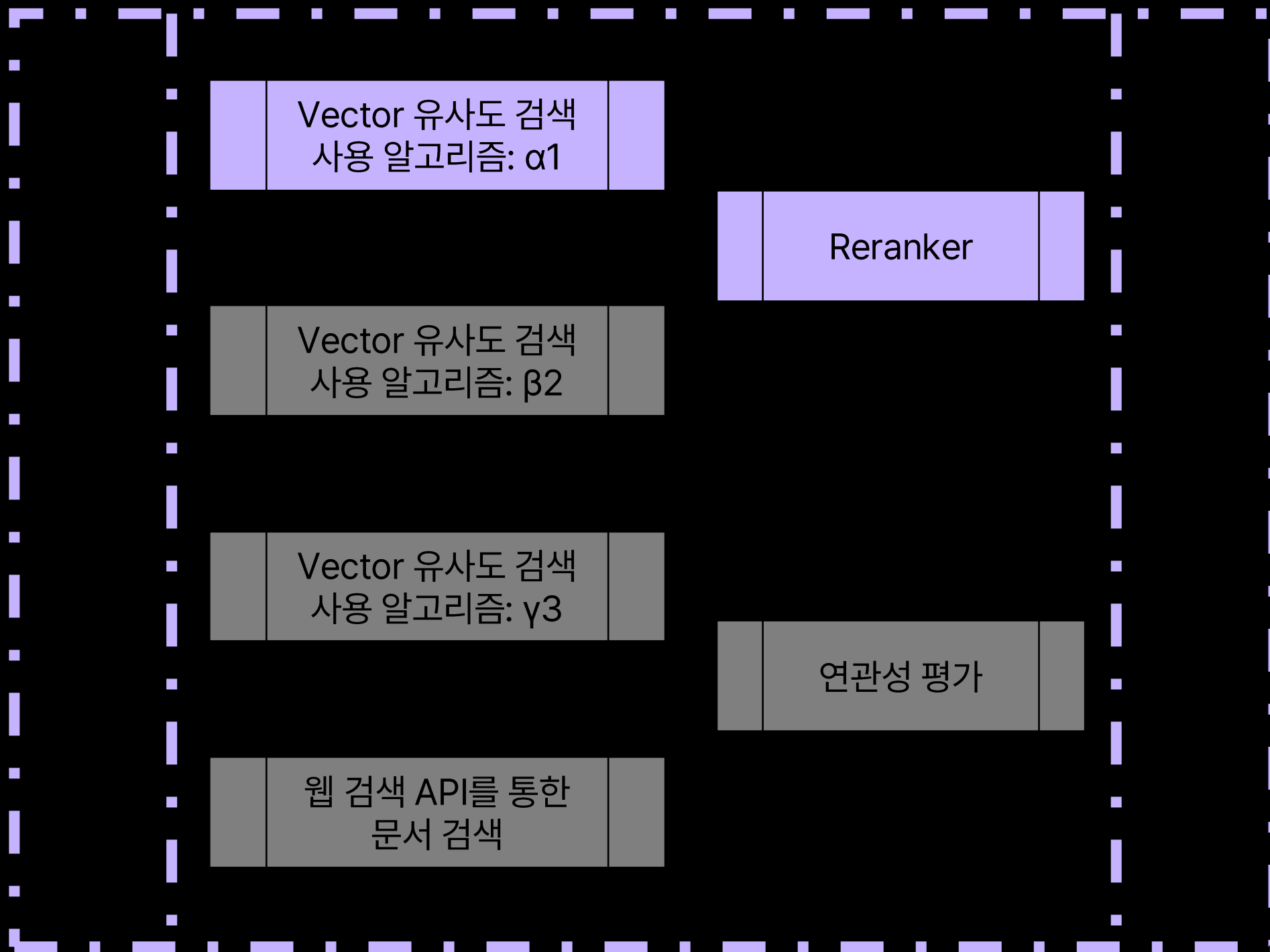
검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

도메인 A에 대한 질문이 들어온 경우:

[α_1 알고리즘 기반 검색] → [웹 검색] → [Reranker]

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

도메인 A에 대한 질문이 들어온 경우:

[α_1 알고리즘 기반 검색] → [웹 검색] → [Reranker]

구현 목적에 따라 웹 검색을 생략할 수도 있고,

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

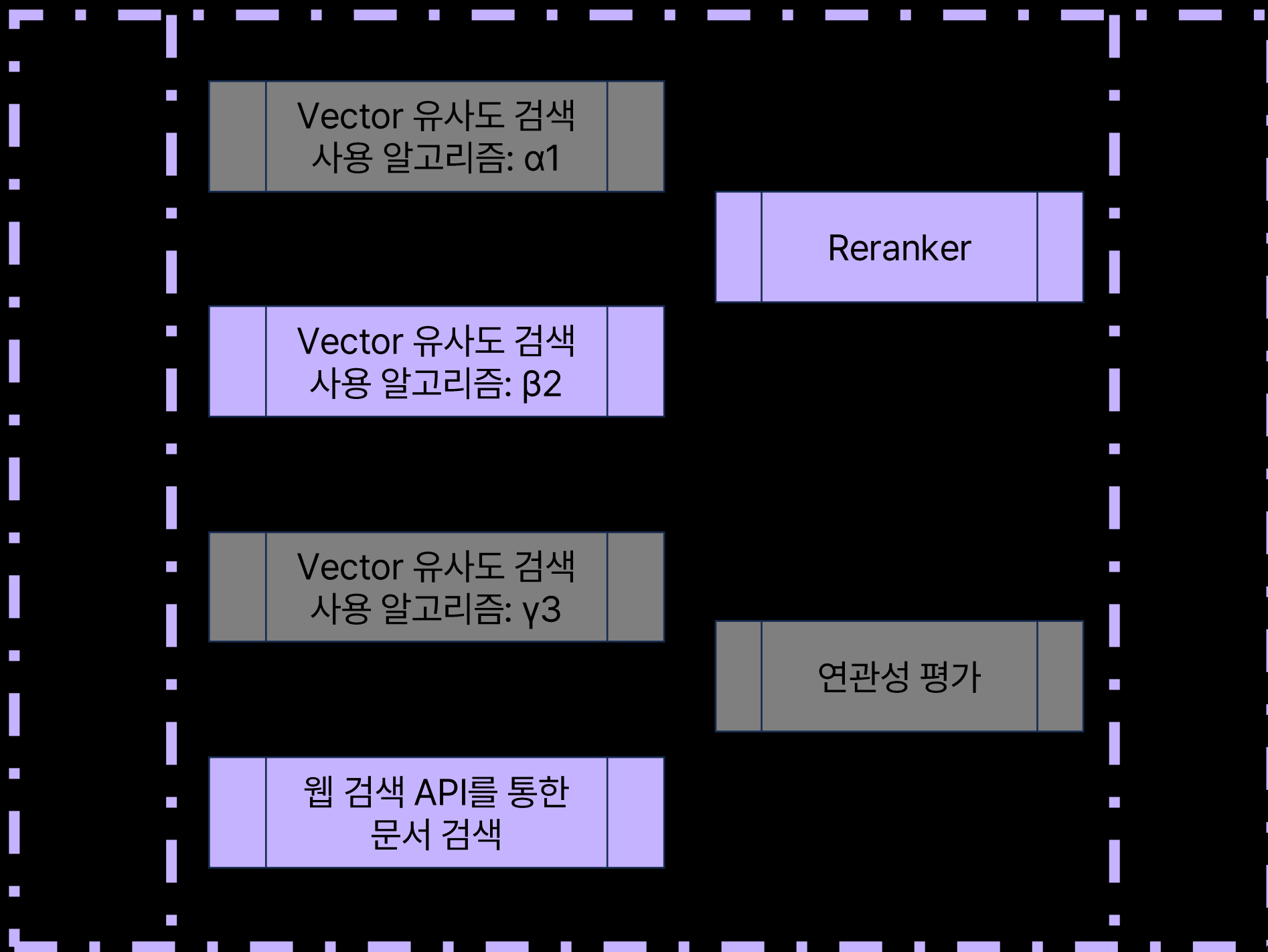
도메인 A에 대한 질문이 들어온 경우:

[α_1 알고리즘 기반 검색] → [웹 검색] → [Reranker]

구현 목적에 따라 웹 검색을 생략할 수도 있고,
[α_1 알고리즘 기반 검색] 결과의 연관성을 평가한 다음 필요에 따라 웹 검색을 수행할 수도 있음

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

도메인 A에 대한 질문이 들어온 경우:

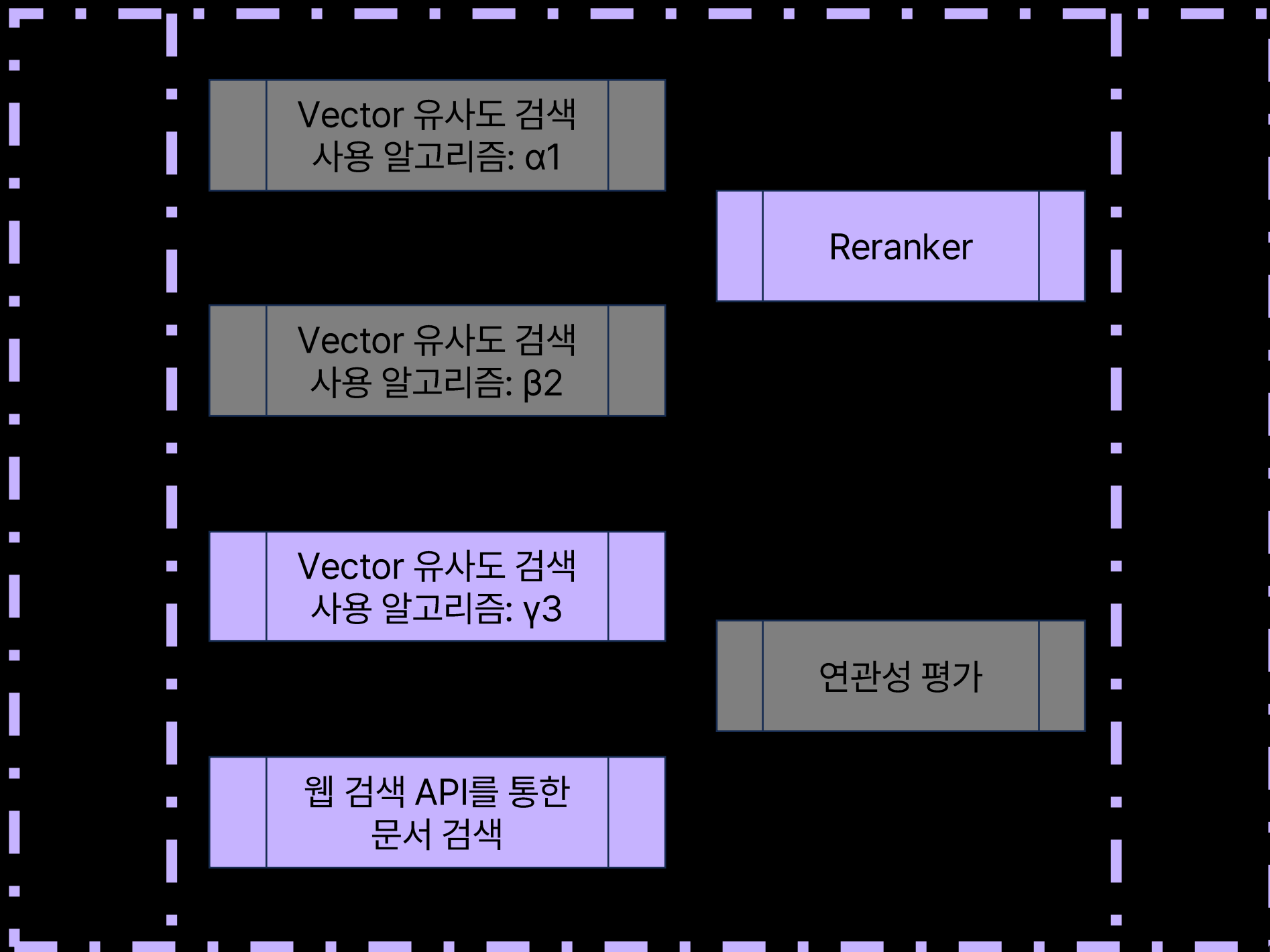
[α_1 알고리즘 기반 검색] → [웹 검색] → [Reranker]

도메인 B에 대한 질문이 들어온 경우:

[β_2 알고리즘 기반 검색] → [웹 검색] → [Reranker]

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를 별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

도메인 A에 대한 질문이 들어온 경우:

[α_1 알고리즘 기반 검색] → [웹 검색] → [Reranker]

도메인 B에 대한 질문이 들어온 경우:

[β_2 알고리즘 기반 검색] → [웹 검색] → [Reranker]

도메인 C에 대한 질문이 들어온 경우:

[γ_3 알고리즘 기반 검색] → [웹 검색] → [Reranker]

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

알 수 없는 질문이 들어온 경우:

[α_1 알고리즘 기반 검색] → [β_2 알고리즘 기반 검색] → [웹 검색] → [γ_3 알고리즘 기반 검색] → [웹 검색] → [Reranker]

모든 가능한 출처에서 정보를 수집한 다음, Reranker를 통해 연관이 있는 정보 위주로 재정렬 하여 제공

Adaptive RAG(On module)

- Adaptive RAG의 핵심 아이디어를
별도의 분류기가 어떤 모듈을 사용할 지 결정하는 방법으로 구현



검색 알고리즘 α_1 , β_2 , γ_3 는 각각 도메인 A, B, C에 특화되었음을 가정

보유한 DB와 관련이 없고, 검색도 필요 없는 경우:
아무런 모듈을 쓰지 않고, NULL(None)을 반환